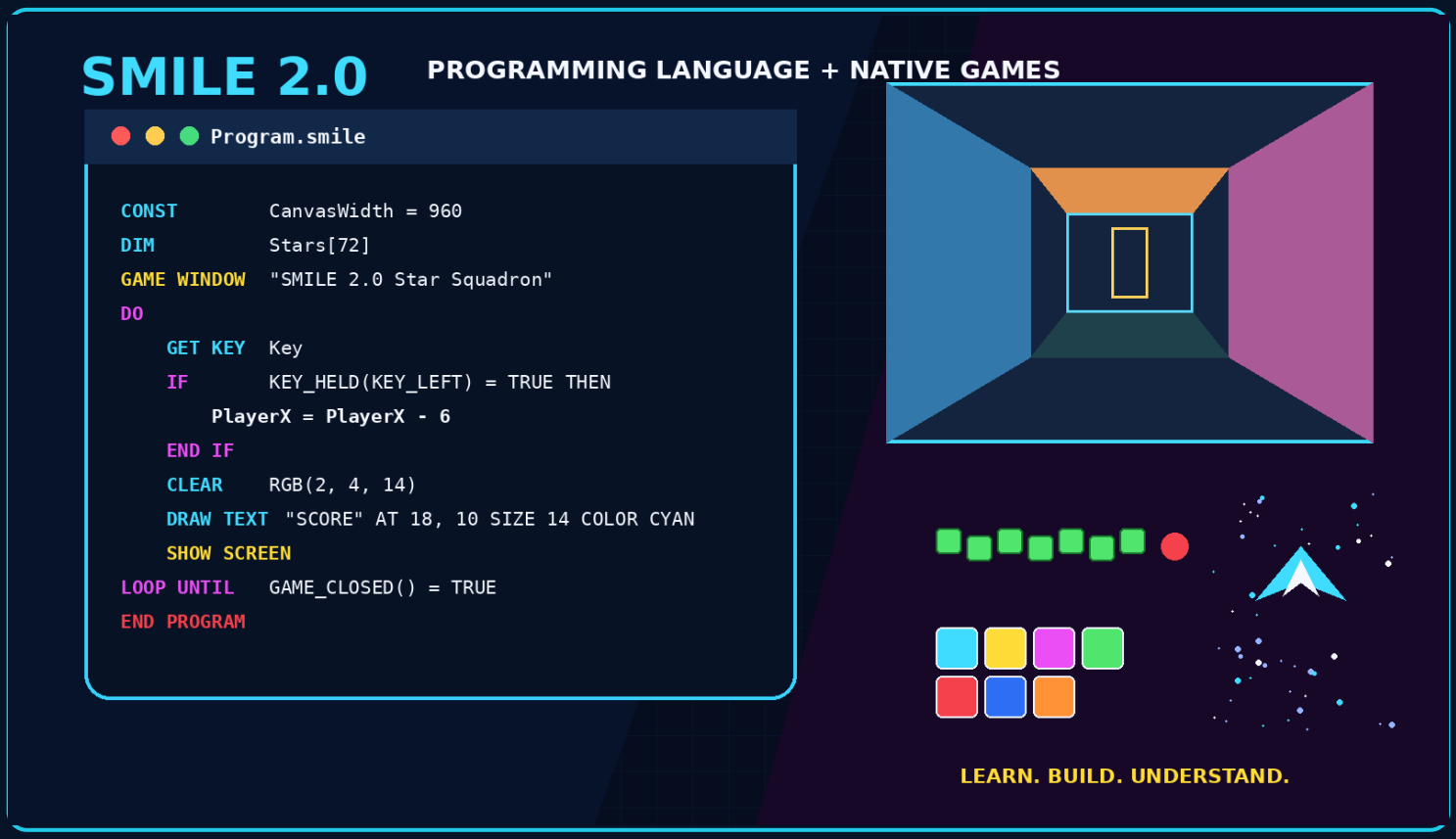
SMILE 2.0

Complete Architecture, Language And Native Games Guide

A first-time student guide to programming-language design and game creation



Learn the language. Build the game. Understand the machine.

Includes all 114 reserved words/constants and all eight committed games.

Packaged version marker: 2.0.1

Exact source snapshot: 92bf27e9d2ae2b06baa486317e3346f80b5d3d83

Created: August 9, 2026 at 4:30:36 PM PHT (UTC+08:00)

Repository: Sincioco/SMILE-2.0

Source-faithful game visuals; reconstruction notice inside

What exact version does this guide describe?

A reproducible source snapshot, not a moving target.

SMILE 2.0

Source-Verified

Explicit packaged version: 2.0.1

Exact Git commit: 92bf27e9d2ae2b06baa486317e3346f80b5d3d83

Latest commit time in this snapshot: August 9, 2026 at 3:52:19 PM PHT (UTC+08:00)

Why both a version and a commit?

The repository exposes 2.0.1 in the Visual Studio extension manifest, but it does not expose a separate semantic version for every compiler/runtime component. The commit hash therefore supplies the most precise identity: it points to one exact set of files.

This guide describes code committed at that snapshot. A later commit can add commands, games or behavior, so future guides should update both the version marker and commit hash.

What counts as evidence?

- Architecture and commands: checked against the shared language source and architecture documentation.
- Game mechanics: checked against each committed README and playable Program-NoDemo.smile source.
- Graphics/audio: checked against implementation reports and runtime source structure.
- Visuals: reconstructed from drawing code because the generated executables target Windows x64 and cannot be run in this Linux PDF environment.

Creation timestamp

This PDF file was generated on August 9, 2026 at 4:30:36 PM PHT (UTC+08:00). The time is shown in Philippine Time because that is the user's current time zone.

Sources: src/Smile.VisualStudio/source.extension.vsixmanifest; repository commit 92bf27e9d2ae2b06baa486317e3346f80b5d3d83

Contents

Page numbers are fixed in this edition.

Recommended reading path

A new student can read pages 4-21 first, choose one game chapter, and return to the keyword reference and glossary whenever a new term appears.

SMILE 2.0 in one sentence

A beginner-friendly BASIC-style language that produces native Windows games.

Think of a programming language as a set of agreements

A language tells humans how to express an idea and tells tools how to understand that idea. SMILE uses readable words such as If, For, Sub, Draw and Play Music. The student writes those ideas in a .smile file.

SMILE 2.0 then analyzes the program, generates Microsoft x64 assembly, links the generic native runtime, and creates a Windows executable. The result does not need Python, .NET or a separate interpreter installed on the player's computer.

The educational goal is not merely to make a toy language. It is to let a student see how variables, decisions, loops, arrays, functions, drawing, sound, files and game algorithms fit together inside one approachable system.

Source basis: README.md and docs/architecture/README.md at snapshot 92bf27e9d2

The design philosophy

Keep the student’s attention on ideas, not plumbing.

Readable, structured BASIC style

SMILE keeps familiar words but uses explicit block endings. A condition is closed by End If; a loop by End For; a routine by End Sub or End Function. That structure makes nesting visible to a beginner.

One obvious way to make a game window

The author declares Game Window, updates state, draws a complete frame, calls Show Screen, and repeats until Game_Closed() becomes True. Fullscreen, scaling, DPI and the selected graphics backend remain runtime responsibilities.

No hidden game engine rules

The native runtime knows how to open a window, read keys, draw shapes, play audio, load files and save integers. It does not secretly implement Snake, falling pieces, enemy AI or raycasting. Those rules remain visible in SMILE source.

Native, not interpreted

Interpreted means another program reads instructions while the program runs. Transpiled means source becomes another high-level language. SMILE 2.0 instead generates MASM x64 and links a native executable.

Project settings do not clutter the language

GraphicsBackend and VSync belong in .smileproj or compiler options, not new student-facing keywords. The drawing language therefore remains stable while implementation choices can evolve.

Safe failure is part of teaching

Map loaders validate dimensions, symbols and connectivity. Missing or invalid map files do not need to crash a lesson: the games can build a deterministic fallback map and explain what happened.

Core rule

SMILE should make the programmer’s idea easy to see. The platform-specific work should be powerful but remain behind a small, teachable language surface.

What has been built so far

The current snapshot is already a complete vertical slice.

Area	What exists now	Why it matters
Language	Structured variables, constants, arrays, decisions, loops, routines, built-in math/time/input, graphics, audio, persistence and text-file loading.	A student can write real programs and complete games without switching languages.
Compiler	Shared analysis plus MASM x64 code generation and Microsoft native linking.	The project teaches the full path from source text to machine-code executable.
Runtime	Win32 window/input, DirectX and GDI graphics, WAV effects, MP3 music, timing, file loading and storage.	Student code stays small while still creating native Windows applications.
Visual Studio 2026	Project system, editor language service, diagnostics, build/run, properties and templates packaged as a VSIX.	SMILE feels like a real language inside a professional IDE.
Games	Eight games, each with demo-enabled and no-demo teaching source, project files, assets and documentation.	The same language concepts are demonstrated across puzzles, action, AI, maps and pseudo-3D.
Quality system	Managed language tests, native graphics/audio tests, smoke builds, asset checks and manual validation documents.	A growing language needs repeatable evidence that new work did not break old work.

Eight complete sample games

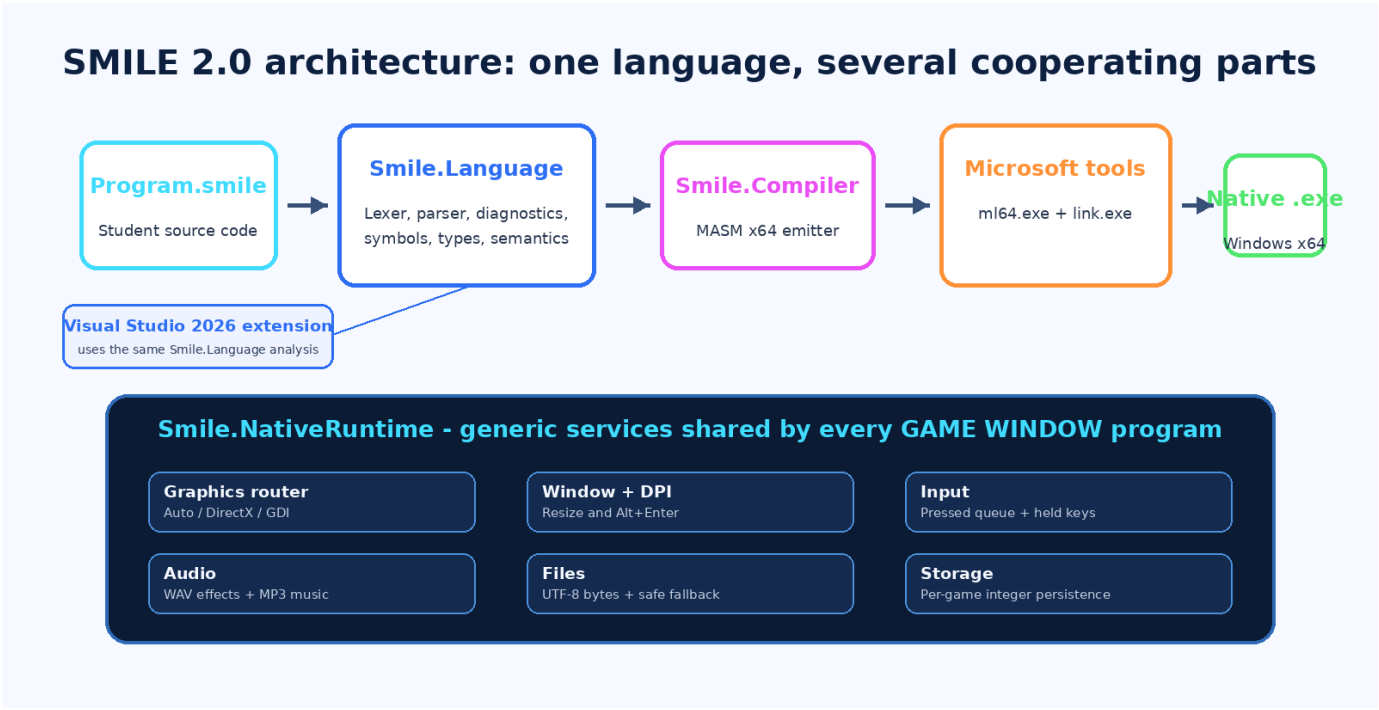
Snake, Falling Blocks, Paddle Ball, Brick Breaker, Dungeon Star I, Dungeon Star II, Maze Muncher and Star Squadron. The game chapters begin on page 38.

A useful way to measure progress

SMILE 2.0 now reaches from language definition to editor, compiler, native runtime, packaged assets, tests and finished games. That end-to-end path is more important than a long list of isolated syntax ideas: every layer has already been exercised by real programs.

Architecture overview

Which part is responsible for which job?



The most important architectural boundary

Smile.Language understands the language. Smile.Compiler turns the understood program into MASM and asks native tools to produce an executable. Smile.NativeRuntime supplies generic services while that executable runs. Smile.VisualStudio presents the same language understanding inside the IDE.

A game source file therefore never needs to know whether a circle is ultimately drawn by Direct2D or GDI. Likewise, Visual Studio does not maintain a second parser that could disagree with the command-line compiler.

Source basis: docs/architecture/README.md; README.md

The repository as a school building

Each folder is a classroom with a clear responsibility.

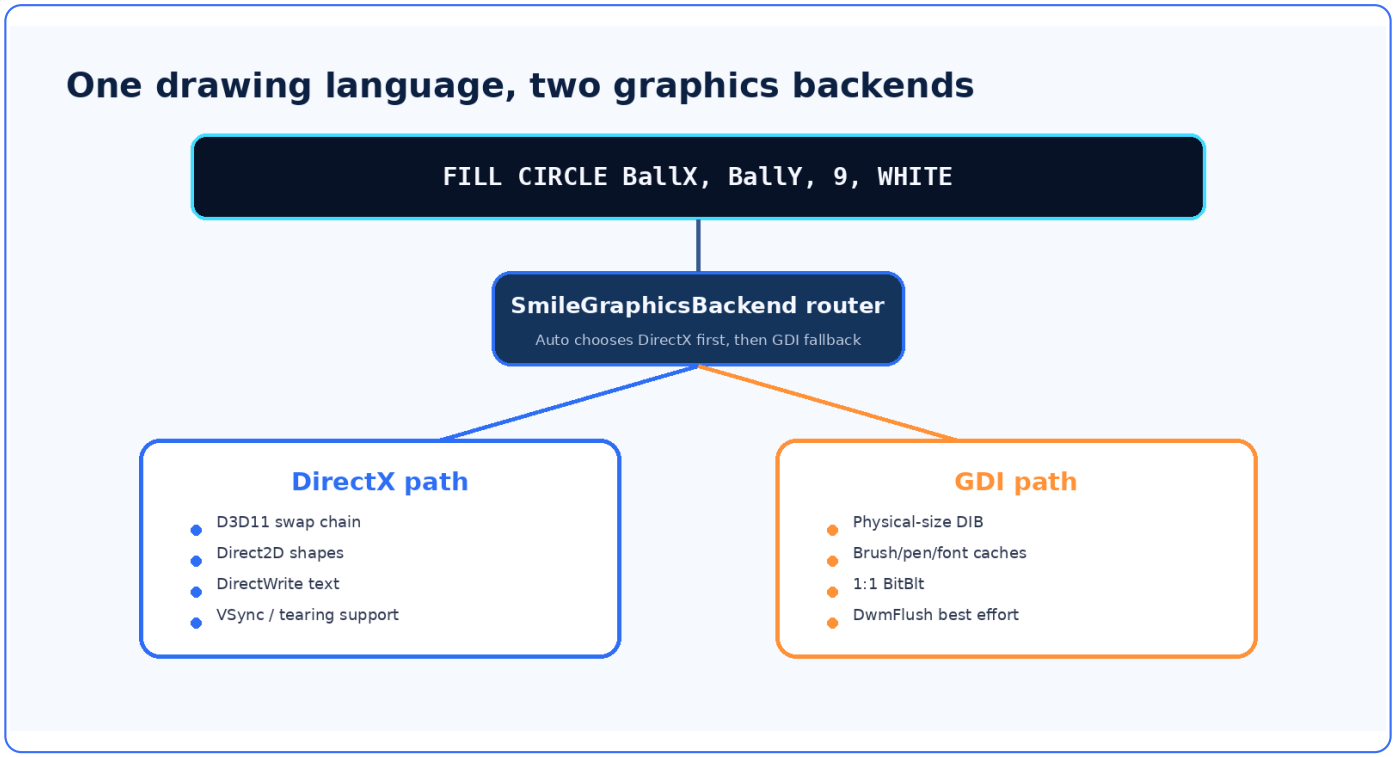
Path	Responsibility	Student-friendly analogy
src/Smile.Language	Lexer, keyword facts, parser, syntax nodes, diagnostics, symbols, types and semantic analysis.	The language teacher who reads and checks every sentence.
src/Smile.Compiler	Compiler driver, MASM emitter, native toolchain invocation and command-line entry point.	The translator and workshop that prepare the final machine instructions.
src/Smile.NativeRuntime	Win32, graphics backends, input, timing, audio, files and storage.	The stage crew: lights, sound, screen, clock and doors.
src/Smile.VisualStudio	Project system, language service, editor diagnostics, properties, templates and VSIX packaging.	The classroom desk where the student writes, builds and runs.
src/Smile.Tests	Language, project and timing tests.	Worksheets that catch mistakes immediately.
src/Smile.NativeGraphicsTests	Native backend, frame state and audio-focus checks.	A safety inspection for platform-specific machinery.
games/	Eight source-visible games, projects, assets, maps and teaching editions.	The laboratory where every feature must prove it is useful.
examples/ docs/ scripts/	Small programs, explanations, build commands, smoke tests and checklists.	Lessons, textbooks and repeatable experiments.

Architecture rule

Adding a new language feature should normally start in Smile.Language, receive code generation in Smile.Compiler, and gain tests and examples before games depend on it.

What happens when a game runs?

Generated code calls a small, stable native service boundary.



Inside the generated executable

1. Startup configures graphics backend and VSync from the project.
2. Game Window initializes the Win32 window and native runtime.
3. The main SMILE loop reads input, updates state and issues draw calls.
4. Show Screen presents one complete frame.
5. End Program and normal exits stop music, release graphics resources and end the process.

The stable C ABI

The MASM emitter calls exported functions with names such as the graphics, sound, music and storage services. A C-compatible binary boundary keeps generated assembly independent from whether the implementation behind that boundary is C, C++ or a Windows component.

This is why the MP3 module can use C++/WinRT internally while the generated SMILE program still sees a small native call.

Source basis: docs/architecture/README.md; direct2d-implementation-report.md; mp3-mediaplayer-implementation-report.md

Four architectural decisions that protect the language

Good boundaries make future features safer.

1. One language authority

The command-line compiler and Visual Studio extension both use Smile.Language. A keyword or diagnostic cannot silently differ between tools.

2. Generic runtime only

The runtime exposes platform services. Snake movement, row clearing, raycasting and enemy AI remain in .smile files a student can inspect.

3. Backend-neutral drawing

Public Draw syntax does not change when Auto selects DirectX or falls back to GDI. Graphics implementation can improve without rewriting games.

4. Settings belong to projects

GraphicsBackend and VSync live in .smileproj or compiler options. The language does not accumulate Windows-specific configuration keywords.

Why these decisions matter to a beginner

A new programmer should be able to ask, “Where does this behavior live?” and receive one answer. Language meaning lives in the language project. Native platform details live in the runtime. Game-specific behavior lives in game source. Tests prove the boundaries still agree.

This separation also makes advanced experiments possible. A future cross-platform runtime, new graphics backend or different native emitter could be explored without first redesigning every game statement.

The compiler pipeline

A sequence of smaller, understandable transformations.

Why use stages?

Trying to translate raw characters directly into machine code would mix spelling, grammar, meaning and CPU rules in one place. SMILE separates those jobs. Each stage receives a cleaner representation than the stage before it.

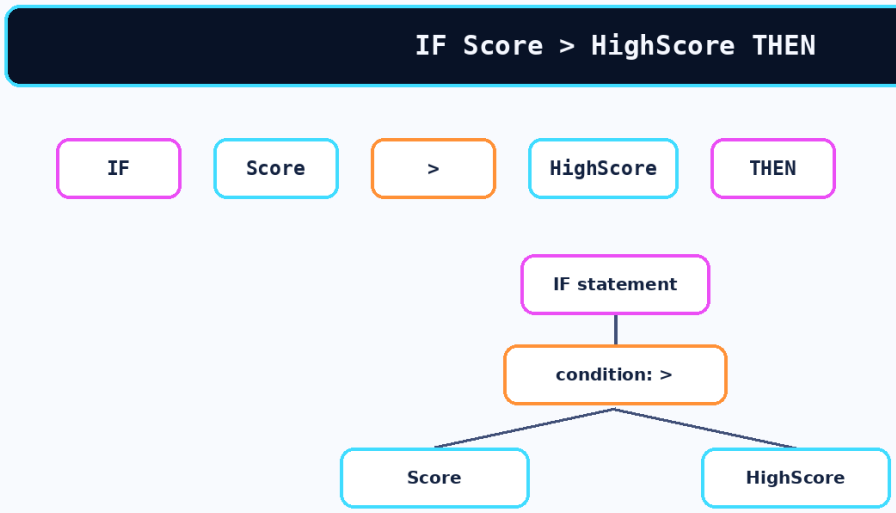
The same analyzed syntax tree can power error messages in Visual Studio and code generation in smilec. That shared model is the reason the IDE can underline the same error that the build reports.

Compiler chapter source: src/Smile.Language; src/Smile.Compiler; docs/architecture/README.md

A tiny example: If Score > HighScore Then

Follow one sentence through the front end.

Tokens and a syntax tree: turning a sentence into a structure



The tree lets later stages reason about the code without rereading raw characters.

Lexer questions

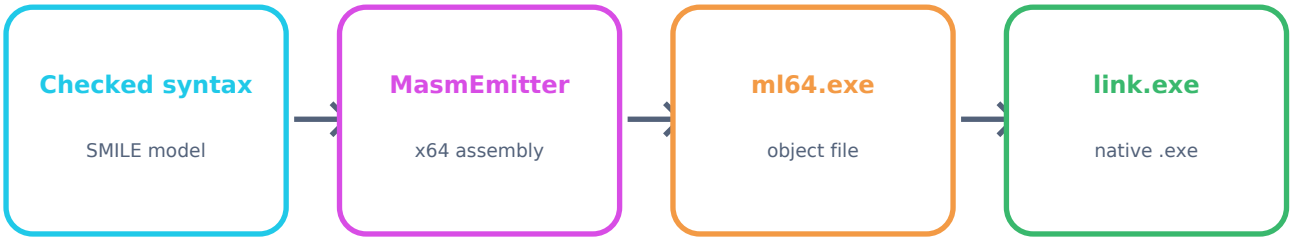
- Where does each word or symbol begin and end?
- Is If a keyword while Score is an identifier?
- Is > a greater-than operator?
- What line and column should an error point to?

Parser and semantic questions

- Does this token order match the If grammar?
- Does the condition produce a Boolean?
- Do Score and HighScore exist?
- Where is the matching End If?

From checked tree to Windows executable

The back end, toolchain and current boundary.



SMILE source

```
Print "Hello from SMILE"
End Program
```

Generated MASM idea

```
; simplified illustration
lea rcx, message_text
call smile_print_text
call smile_print_newline
xor ecx, ecx
call ExitProcess
```

What the toolchain adds

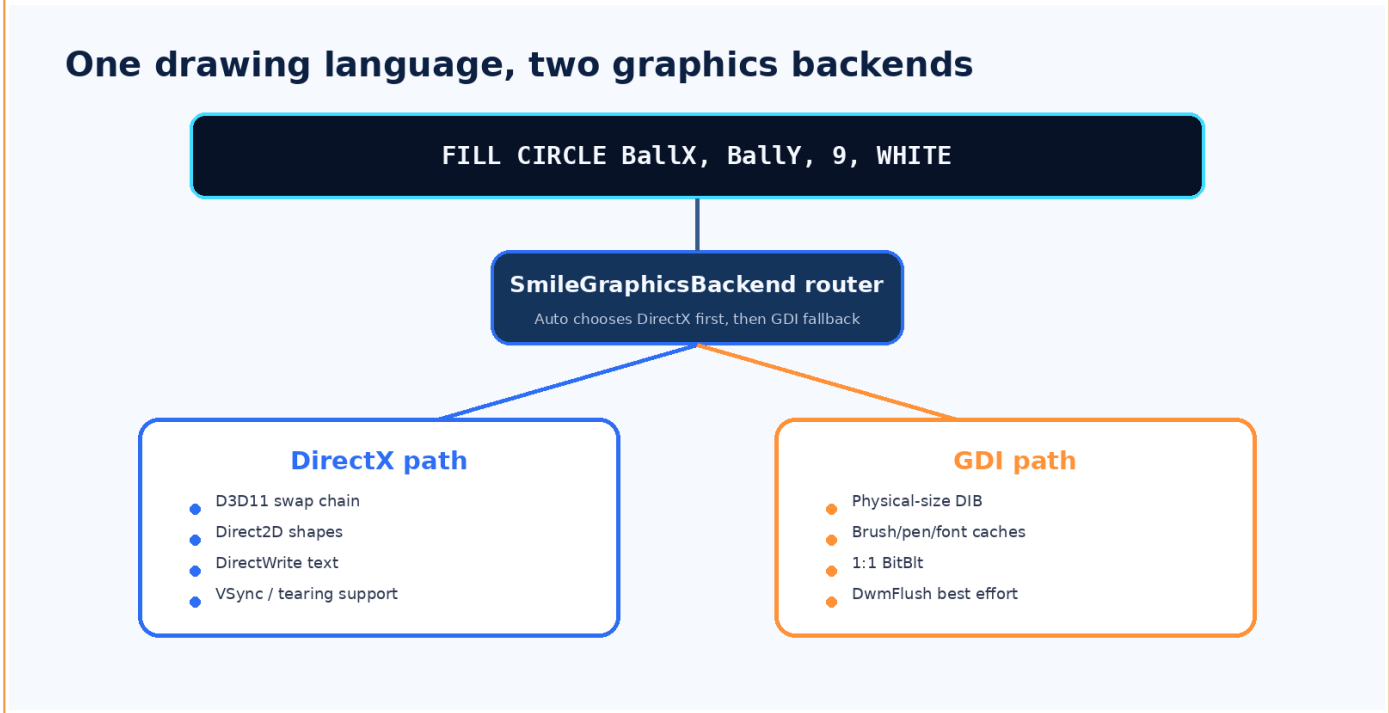
The linker combines generated object code, the SMILE runtime library and Windows import libraries. Console-only programs pull only the services they reference. A music-bearing program also receives the MediaPlayer-related runtime dependencies.

Current compiler boundary

The language currently stores signed 64-bit integers, uses fixed-size arrays up to two dimensions, supports routines with up to four scalar parameters, and uses literal-oriented text features. Those are deliberate current limits, not claims that the architecture can never grow.

Graphics: DirectX first, GDI maintained

The same public drawing commands reach either backend.



Term	DirectX path	GDI path
Device/presentation	D3D11 hardware device and two-buffer flip-discard swap chain.	Physical client-size DIB and 1:1 BitBlt presentation.
Shapes	Direct2D 1.1 geometry and brush cache.	Pens, brushes and physical-resolution mapping.
Text	DirectWrite layouts and grayscale anti-aliasing.	Cached fonts with ClearType/anti-aliased fallbacks.
Selection	Auto tries DirectX; explicit DirectX never silently changes.	Auto fallback after DirectX failure; explicit GDI skips DirectX.

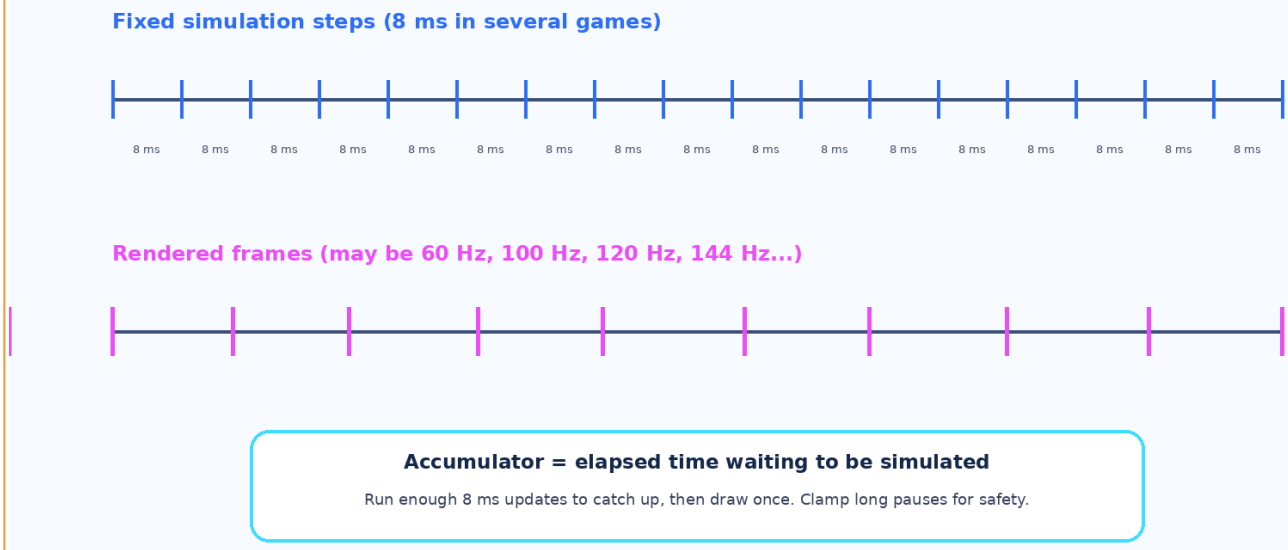
Student-facing promise

Coordinates remain in the logical canvas. Resize, fullscreen, physical DPI, clipping and backend selection are handled without changing the game’s Draw statements.

Timing, input, fullscreen and DPI

Why the ball should not move faster on a 144 Hz monitor.

Game timing: simulation and drawing are related, but not identical



Accumulator = elapsed time waiting to be simulated

Run enough 8 ms updates to catch up, then draw once. Clamp long pauses for safety.

Two input questions

Get Key asks, “What key was pressed?” It is useful for menus, fire buttons and one-time actions.

Key_Held(key) asks, “Is this key still down?” It is useful for smooth movement.

Keeping both ideas prevents a held movement key from flooding a menu and prevents continuous motion from depending on keyboard repeat settings.

Window behavior the student does not code

- Alt+Enter toggles borderless fullscreen.
- The logical canvas scales uniformly.
- Non-16:9 space becomes black letterbox bars.
- Resize, minimize/restore and DPI changes rebuild backend resources safely.
- VSync and frame-latency pacing are project/runtime concerns.

Audio, assets, storage and map files

Four different kinds of data, four different teaching jobs.

Feature	SMILE syntax	Runtime behavior	Teaching purpose
WAV effect	Play Sound "hit.wav" Stop Sound	One asynchronous effect channel using executable-relative paths.	Immediate feedback for hits, food, scores and transitions.
MP3 music	Play Music ... Loop Pause/Resume/Stop Music Music Volume n	One MediaPlayer channel. Focus loss effectively mutes without changing another app’s volume.	Background music lifecycle and focus-aware behavior.
Persistence	Load value From key Default n Save value To key	Per-game integer storage under safe runtime-controlled paths.	High scores and simple settings that survive a run.
UTF-8 file bytes	Load Text File path Into array Count length	Loads executable-relative bytes into a fixed array; missing data is nonfatal.	Student-defined maps, parsing, validation and fallback design.

Important path rule

Relative asset paths are based on the generated executable’s directory, not whichever folder happens to be the process working directory.

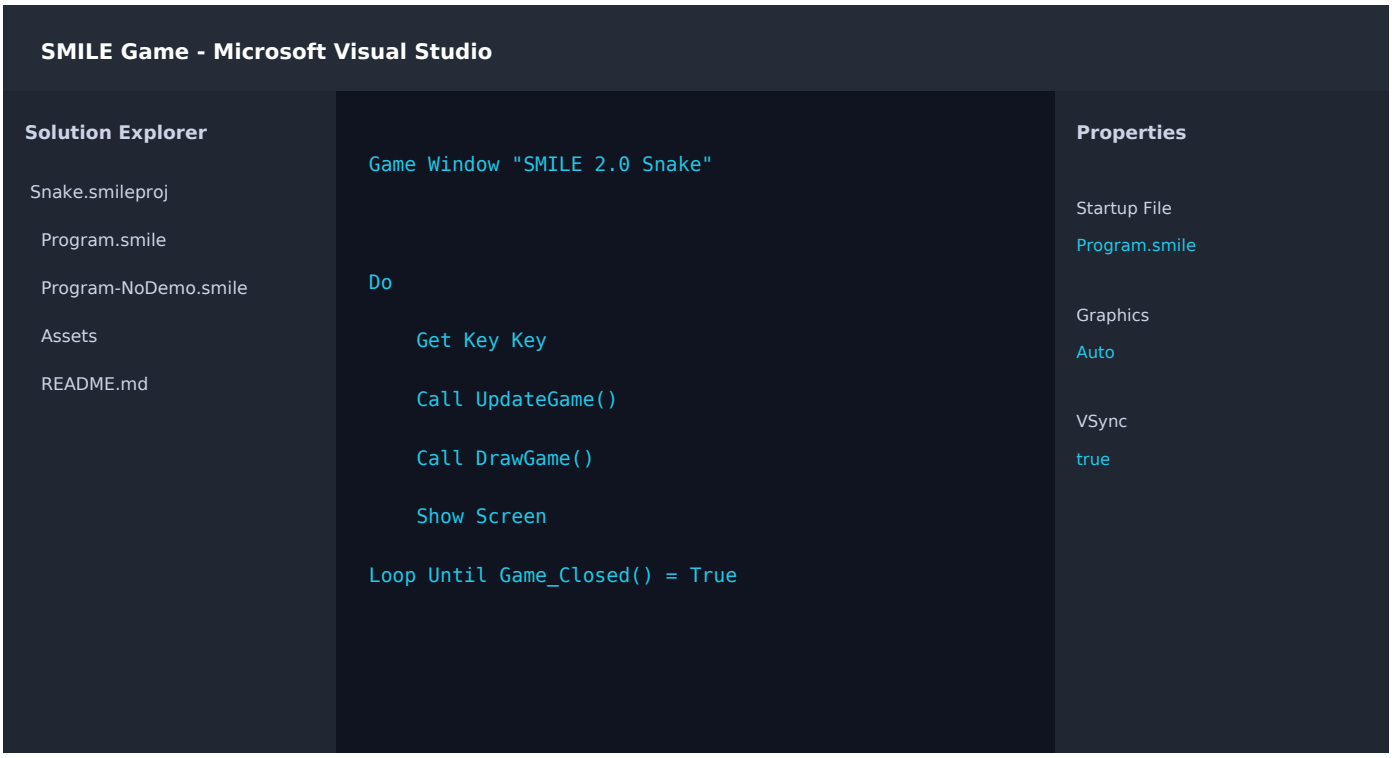
Typical data and audio lifecycle

```
Load Text File "Maps\default.map"
    Into MapFileBytes Count MapFileLength
If MapValid = False Then
    Call BuildFallbackMap()
End If

Music Volume 38
Play Music "Assets\Background.mp3" Loop
```

Visual Studio 2026 integration

SMILE is not limited to a command prompt.



What the VSIX supplies

- A SMILE project type and templates.
- Syntax-aware editor classification and diagnostics.
- Build and run commands that use the same compiler.
- Solution Explorer items for source and assets.
- Project properties for StartupFile, GraphicsBackend and VSync.

The consistency guarantee

The extension does not implement its own private version of SMILE. It references the same Smile.Language assembly used by the native compiler path. A diagnostic seen while editing should therefore match the diagnostic produced during build.

Packaging

The VSIX manifest identifies the current packaged extension as version 2.0.1. Templates and required runtime/compiler payloads are validated by the build and smoke-test scripts.

Testing and governance

A language change is not finished when it merely compiles once.

Language tests
Lexing, parsing, diagnostics, semantic rules, project options and timing calculations.

Native tests
Backend selection, fallback, frame invalidation, viewport/DPI calculations and audio-focus state.

Smoke suite
Build compiler, runtime, tests, examples, all games, assets, templates and VSIX; inspect generated executables.

Manual validation
Windowed/fullscreen, resize, minimize/restore, monitor movement, visual sharpness, audio and long-running resource stability.

Why governance files matter

A growing language can accidentally gain duplicate parsers, game-specific native shortcuts or inconsistent syntax. Permanent rules and Codex guardrails turn architectural intent into a review checklist: read the authority documents, change the correct layer, add tests, run the full suite, and commit a reproducible result.

For a student, this demonstrates an important professional lesson: software quality is a process, not a final “test button.”

Values, variables, constants and arrays

The memory model a beginner sees.

Variables appear through assignment

SMILE does not require a separate declaration for every scalar variable. The first assignment establishes the name in the program's analyzed symbol model.

Score = 0
PlayerX = CanvasWidth / 2
Ready = False

Constants explain important numbers

Const CellSize = 22 is easier to understand and change than repeating 22 throughout a game. Constants can use integer expressions involving earlier constants.

Current numeric model

The current language stores signed 64-bit integer values. Booleans and color/key constants are represented within that practical numeric model. Games needing fractions use fixed-point scaling.

Arrays reserve indexed storage

Dim SnakeX[840] reserves one dimension. Dim Board[10, 20] reserves two. Sizes are fixed at compile time, which keeps native storage predictable.

An index selects one element:
Board[Column, Row] = PieceValue

Why arrays are central to games

- A board is a grid of cells.
- A snake is an ordered list of coordinates.
- A projectile pool is a set of reusable slots.
- A map file is a byte array plus a length.
- Raycasting stores one result per screen ray.

Bounds matter

Game code normally tests X/Y ranges before indexing. Map helper functions return a wall for out-of-range coordinates, turning a dangerous case into a safe rule.

Data example

```
Const GridWidth = 35
Const GridHeight = 24
Const MaximumCells = GridWidth * GridHeight

Dim SnakeX[MaximumCells]
Dim SnakeY[MaximumCells]

SnakeX[0] = GridWidth / 2
SnakeY[0] = GridHeight / 2
```

Expressions, decisions and loops

How a program chooses and repeats.

Three control-flow patterns

```
If Lives <= 0 Then
    State = STATE_GAME_OVER
Else If PelletCount = 0 Then
    State = STATE_LEVEL_CLEAR
Else
    Call UpdatePlayer()
End If

For Row = 0 To Height - 1
    For Column = 0 To Width - 1
        Call DrawCell(Column, Row)
    End For
End For

Do
    ... process queue ...
Loop Until QueueHead >= QueueTail
```

Expressions make values

Arithmetic, comparisons, Boolean operators, parentheses, function calls, array reads and literals combine into expressions. Integer division truncates toward zero; Mod returns a remainder.

Conditions are explicit

An If or Loop Until condition must be a Boolean expression. Games commonly compare a variable with True or False so the intended meaning is visible.

Choose the loop that explains the job

- For: a known count or range.
- For ... Down To: safe backward shifting/collapse.
- Do / Loop: repeat until code exits.
- Do / Loop Until: stop when a bottom condition becomes true.
- Exit For / Exit Do: leave early after finding an answer.

Select Case

Select Case is clearer than a long chain of equality tests when one expression chooses among several fixed values. Falling Blocks uses it for piece colors and line-clear score tables; Brick Breaker uses it for row colors.

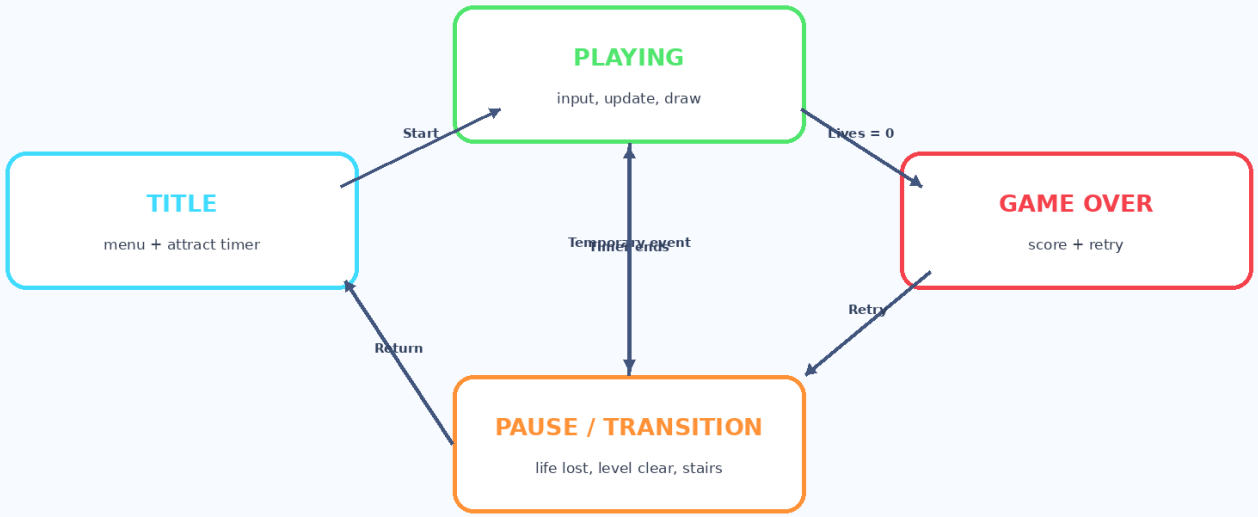
Integer-division example

With signed integers, 7 / 3 becomes 2. Games intentionally multiply before dividing when they want a scaled fraction: Position + Speed * Step / 1000.

Routines and the game loop

Organize a large program into named ideas.

A game state machine: only one major screen owns control at a time



Sub: perform an action

A Sub groups statements but does not produce a result. Examples: ResetGame, DrawPlayer, UpdateEnemies, BuildFallbackMap.

Call makes the action visible at the use site: Call DrawGame()

Function: calculate an answer

A Function returns a value. Examples: CanMove, CellIndex, EnemyColor and WallColorFor. Current routines accept up to four scalar parameters.

The continuous game loop

Most games repeat the same broad cycle:

1. Read pressed and held input.
2. Update timers and game state.
3. Draw the frame into the back buffer.
4. Show Screen.
5. Stop when Game_Closed() is True.

A state variable decides whether the loop is currently updating a title screen, gameplay, a transition or game over.

Every reserved word - reference 1 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Dim	Data	Reserves a fixed one- or two-dimensional numeric array.	Dim Board[10, 20] - indexed storage for maps, pieces and enemies.
Const	Data	Creates a named integer value that cannot be reassigned.	Const CellSize = 20 - gives an important number a readable name.
True	Boolean	Built-in Boolean value meaning yes/on/condition succeeded.	If Ready = True Then ...
False	Boolean	Built-in Boolean value meaning no/off/condition failed.	Collision = False
If	Decision	Starts a conditional block. Its expression must evaluate to a Boolean.	If Score > HighScore Then
Then	Decision	Separates an If condition from the statements that run when it is true.	If Lives = 0 Then
Else	Decision	Introduces the alternate path. It also forms Else If.	Else If Key = KEY_ESCAPE Then
End	Block closing	Closes a structured block when paired with another word.	End If, End For, End Sub, End Function, End Select, End Program.
And	Logic	True only when both Boolean conditions are true.	Key = KEY_W And Direction <> KEY_DOWN
Or	Logic	True when either Boolean condition is true.	Key = KEY_A Or Key = KEY_LEFT

Every reserved word - reference 2 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Not	Logic	Reverses a Boolean value.	Not GameOver - useful when a negative test reads more clearly.
For	Loop	Starts a counted loop with a predictable range.	For Index = 0 To 9
To	Range / destination	Marks the ending value of a For range and appears in Save ... To.	For Row = 0 To 19; Save HighScore To "HighScore".
Down	Loop / constant	Makes a For loop count downward. It also doubles as short direction constant 11.	For I = Length - 1 Down To 1
Do	Loop	Starts a general-purpose loop.	Do ... Loop or Do ... Loop Until Ready = True.
Loop	Loop / music	Closes Do. After Play Music it requests continuous playback.	Loop Until Done = True; Play Music "song.mp3" Loop.
Until	Loop	Provides the stop condition at the bottom of a Do loop.	Loop Until QueueHead >= QueueTail
Exit	Loop control	Leaves the nearest For or Do loop early.	Exit Do; Exit For.
Select	Decision	Starts a multi-way choice based on one expression.	Select Case PieceValue
Case	Decision	Introduces one branch inside Select; Case Else is the fallback.	Case 1 ... Case Else ... End Select.

Every reserved word - reference 3 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Print	Console output	Writes one or more values to the console. A final semicolon suppresses the newline.	Print "Score: ", Score
Get	Input	Begins Get Key, which removes the next pressed key from the input queue.	Get Key Key
Key	Input	Completes Get Key. The destination variable receives a key constant.	Get Key Key
Clear	Console / graphics	Clears the console with Clear Screen or fills the game canvas with Clear color.	Clear Screen; Clear Rgb(8, 12, 22).
Screen	Console / presentation	Completes Clear Screen and Show Screen.	Show Screen presents the completed back buffer.
Wait	Timing	Pauses execution for a requested duration.	Wait 500 Milliseconds
Milliseconds	Timing	Names the unit used by Wait.	Wait 100 Milliseconds
Random	Randomness	Assigns an inclusive random integer to a variable.	Random FoodX From 0 To GridWidth - 1
From	Range / persistence	Marks a random lower bound or a persistence source.	Random X From 1 To 10; Load Score From "HighScore" Default 0.
Mod	Arithmetic	Returns the integer remainder after division.	Direction = (Direction + 1) Mod 4

Every reserved word - reference 4 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Sub	Routine	Declares a procedure that performs work but does not return a value.	Sub DrawBoard() ... End Sub
Call	Routine	Invokes a Sub routine.	Call ResetGame()
Function	Routine	Declares a routine that computes and returns an integer/Boolean value.	Function CanMove(X, Y, Rotation) ... End Function.
Return	Routine	Leaves a routine; in a Function it supplies the result.	Return True; Return CellIndex(X, Y).
Program	Program control	Used with End to terminate the generated process immediately and cleanly.	End Program
Timer	Built-in function	Returns elapsed milliseconds from the runtime clock.	NextMoveTime = Timer() + 100
Rgb	Built-in function	Combines red, green and blue components into a drawable color value.	Rgb(40, 220, 30)
Abs	Built-in function	Returns the non-negative magnitude of an integer.	Abs(BallX - EnemyX)
Min	Built-in function	Returns the smaller of two integers.	Min(50, Elapsed)
Max	Built-in function	Returns the larger of two integers.	Max(80, 700 - Level * 60)

Every reserved word - reference 5 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Game_Closed	Built-in function	Returns True after the user requests that the game window close.	Loop Until Game_Closed() = True
Key_Held	Built-in function	Reports whether a key is currently held down, not merely pressed once.	If Key_Held(KEY_LEFT) = True Then
Game	Graphics	Starts the game-window declaration. A program using graphics has one Game Window.	Game Window "My Game"
Window	Graphics	Completes Game Window and supplies the title.	Game Window "SMILE 2.0 Snake" Size 960 By 540.
Size	Graphics / text	Sets a game window logical size or the font size of drawn text/numbers.	Size 960 By 540; Draw Text ... Size 24.
By	Graphics	Separates width and height in a Game Window Size clause.	Size 960 By 540
Fill	Graphics	Requests a solid shape rather than only an outline.	Fill Circle X, Y, 9, YELLOW
Draw	Graphics	Starts an outlined shape, line, text or number command.	Draw Rectangle ...; Draw Text ...
Rectangle	Graphics	Names an axis-aligned rectangle primitive.	Fill Rectangle X, Y, Width, Height, Color.
Rounded	Graphics	Combines with Rectangle to add a corner-radius argument.	Draw Rounded Rectangle X, Y, W, H, Radius, Color.

Every reserved word - reference 6 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Circle	Graphics	Names a circle primitive using center X/Y and radius.	Fill Circle BallX, BallY, BallRadius, WHITE.
Arc	Graphics	Draws part of a circle outline using start and sweep angles.	Draw Arc X, Y, Radius, StartAngle, SweepAngle, Color.
Quadrilateral	Graphics	Names a four-corner polygon used for ships, walls and perspective panels.	Fill Quadrilateral x1,y1,x2,y2,x3,y3,x4,y4,Color.
Line	Graphics	Draws a straight line between two points.	Draw Line X1, Y1, X2, Y2, Color.
Text	Graphics / files	Selects literal text drawing, and also appears in Load Text File.	Draw Text "Ready" At ...; Load Text File "map" Into Bytes Count Length.
Number	Graphics	Draws the current value of a numeric expression.	Draw Number Score At 100, 50 Size 32 Color YELLOW.
At	Graphics	Introduces the X/Y position for Draw Text or Draw Number.	Draw Text "Score" At 18, 10 ...
Color	Graphics	Introduces the color expression for text or numbers.	Size 24 Color CYAN
Centered	Graphics	Centers drawn text horizontally around its X coordinate.	Draw Text "Game Over" At 480, 200 ... Centered.
Show	Presentation	Begins Show Screen, which presents the completed frame.	Draw everything first, then Show Screen once.

Every reserved word - reference 7 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Play	Audio	Starts a WAV effect or MP3 music command.	Play Sound "hit.wav"; Play Music "background.mp3" Loop.
Sound	Audio	Selects the single asynchronous WAV effect channel.	Play Sound "Assets\Eat.wav"; Stop Sound.
Music	Audio	Selects the single MP3 background-music channel.	Play, Pause, Resume, Stop, or set Music Volume.
Pause	Audio	Temporarily pauses the active music channel without losing its position.	Pause Music
Resume	Audio	Continues music that was manually paused.	Resume Music
Volume	Audio	Sets requested music volume from 0 through 100.	Music Volume 38
Stop	Audio	Stops the current WAV effect or MP3 music channel.	Stop Sound; Stop Music.
Load	Storage / files	Loads a persistent integer or executable-relative UTF-8 file bytes.	Load HighScore From ...; Load Text File ...
File	File input	Completes Load Text File.	Load Text File "Maps\default.map" Into Bytes Count Length.
Into	File input	Names the fixed numeric array that receives file bytes.	... Into MapFileBytes ...

Every reserved word - reference 8 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
Count	File input	Names the variable that receives the number of bytes loaded.	... Count MapFileLength
Save	Persistence	Writes an integer under a per-game storage key.	Save HighScore To "HighScore"
Default	Persistence	Supplies the value used when a saved key is missing or invalid.	Load HighScore From "HighScore" Default 0
NONE	Short input constant	Numeric value 0, meaning no key/direction.	KEY_NONE is the clearer named form used by most games.
W	Short input constant	Numeric value for the W key.	Useful in simple input comparisons; the identifier W is reserved.
A	Short input constant	Numeric value for the A key.	Short form corresponding to KEY_A.
S	Short input constant	Numeric value for the S key.	Short form corresponding to KEY_S.
D	Short input constant	Numeric value for the D key.	Short form corresponding to KEY_D.
UP	Short input constant	Numeric value for the Up Arrow key.	Short form corresponding to KEY_UP.
LEFT	Short input constant	Numeric value for the Left Arrow key.	Short form corresponding to KEY_LEFT.

Every reserved word - reference 9 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
RIGHT	Short input constant	Numeric value for the Right Arrow key.	Short form corresponding to KEY_RIGHT.
KEY_NONE	Key constant	No queued key press.	Use to reset or test an input variable.
KEY_W	Key constant	W key.	Common move-forward or player-one-up control.
KEY_A	Key constant	A key.	Common move-left or turn-left control.
KEY_S	Key constant	S key.	Common move-back/down control.
KEY_D	Key constant	D key.	Common move-right or turn-right control.
KEY_UP	Key constant	Up Arrow key.	Alternative movement/selection control.
KEY_DOWN	Key constant	Down Arrow key.	Alternative movement/selection control.
KEY_LEFT	Key constant	Left Arrow key.	Alternative movement/selection control.
KEY_RIGHT	Key constant	Right Arrow key.	Alternative movement/selection control.

Every reserved word - reference 10 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
KEY_ENTER	Key constant	Enter key.	Start, confirm, retry or open a door.
KEY_ESCAPE	Key constant	Escape key.	Return to title or exit.
KEY_SPACE	Key constant	Spacebar.	Fire, hard drop, start or interact.
KEY_1	Key constant	Number 1 key.	Mode selection, such as one-player Paddle Ball.
KEY_2	Key constant	Number 2 key.	Mode selection, such as two-player Paddle Ball.
KEY_OTHER	Key constant	A pressed key that is recognized but has no dedicated named constant.	Useful for “any key” behavior.
BLACK	Color constant	Built-in black drawing color.	Use anywhere a graphics command expects a color: ... Color BLACK.
WHITE	Color constant	Built-in white drawing color.	Use anywhere a graphics command expects a color: ... Color WHITE.
RED	Color constant	Built-in red drawing color.	Use anywhere a graphics command expects a color: ... Color RED.
GREEN	Color constant	Built-in green drawing color.	Use anywhere a graphics command expects a color: ... Color GREEN.

Every reserved word - reference 11 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
BLUE	Color constant	Built-in blue drawing color.	Use anywhere a graphics command expects a color: ... Color BLUE.
CYAN	Color constant	Built-in cyan drawing color.	Use anywhere a graphics command expects a color: ... Color CYAN.
MAGENTA	Color constant	Built-in magenta drawing color.	Use anywhere a graphics command expects a color: ... Color MAGENTA.
YELLOW	Color constant	Built-in yellow drawing color.	Use anywhere a graphics command expects a color: ... Color YELLOW.
ORANGE	Color constant	Built-in orange drawing color.	Use anywhere a graphics command expects a color: ... Color ORANGE.
GRAY	Color constant	Built-in gray drawing color.	Use anywhere a graphics command expects a color: ... Color GRAY.
DARK_RED	Color constant	Built-in dark red drawing color.	Use anywhere a graphics command expects a color: ... Color DARK_RED.

Every reserved word - reference 12 of 12

114 unique reserved spellings in the current shared lexer: 73 general language words and 41 additional constant words. Down also doubles as a direction constant.

Word	Category	What it does	Why it is needed / example
DARK_GREEN	Color constant	Built-in dark green drawing color.	Use anywhere a graphics command expects a color: ... Color DARK_GREEN.
DARK_BLUE	Color constant	Built-in dark blue drawing color.	Use anywhere a graphics command expects a color: ... Color DARK_BLUE.
DARK_GRAY	Color constant	Built-in dark gray drawing color.	Use anywhere a graphics command expects a color: ... Color DARK_GRAY.
LIGHT_RED	Color constant	Built-in light red drawing color.	Use anywhere a graphics command expects a color: ... Color LIGHT_RED.
LIGHT_GREEN	Color constant	Built-in light green drawing color.	Use anywhere a graphics command expects a color: ... Color LIGHT_GREEN.
LIGHT_BLUE	Color constant	Built-in light blue drawing color.	Use anywhere a graphics command expects a color: ... Color LIGHT_BLUE.
LIGHT_GRAY	Color constant	Built-in light gray drawing color.	Use anywhere a graphics command expects a color: ... Color LIGHT_GRAY.

Operators and punctuation are not keywords

They still form the grammar of expressions and statements.

Symbol / form	Meaning	Example and note
+ - * /	Integer arithmetic. Unary + and - are also supported.	Distance = Abs(X2 - X1); integer division truncates.
Mod	Remainder after integer division.	(Direction + 1) Mod 4 wraps four directions.
= <> < > <= >=	Equality and ordering comparisons.	Score > HighScore; Tile <> TILE_WALL.
And Or Not	Boolean logic.	Inside And range checks, Or key alternatives, Not negation.
( )	Grouping and routine/function argument lists.	Max(0, Value); (A + B) * C.
[ ]	Array dimensions and indexes.	Dim Board[10,20]; Board[X,Y].
,	Separates arguments, coordinates and Print items.	Rgb(40,220,30); Print "Score", Score.
;	At the end of Print, suppresses the newline.	Print "Loading"; - later output stays on that line.
'	Starts a comment through the end of the line.	' This explains why the next constant exists.
"..."	Text literal. Two adjacent quotes inside represent a quote character.	Print "He said ""Hello""."

Case-insensitive language

Print, Print and print are recognized as the same keyword. Consistent capitalization is still valuable because it makes examples easier to scan.

Precedence still matters

Parentheses make intention clear. Prefer (A + B) * C when a reader should not need to remember operator precedence.

Core console and structured-language recipes

Small patterns that combine the individual keywords.

Data, output, input and If

```
Const MaximumLives = 3
Dim Scores[10]

Print "What is your score?"
Get Key Key

If Key = KEY_ESCAPE Then
    End Program
End If
```

Sub and Function

```
Sub AddScore(Points)
    Score = Score + Points
End Sub

Function IsInside(X, Minimum, Maximum)
    Return X >= Minimum And X <= Maximum
End Function

Call AddScore(100)
```

Formatting rule

SMILE is line-oriented. Put one clear statement on a line, indent the body of structured blocks, and use blank lines to separate ideas.

For, Do, Random and Select Case

```
For Index = 0 To 9
    Scores[Index] = 0
End For

Do
    Random Value From 1 To 6
Loop Until Value = 6

Select Case Value
    Case 6
        Print "Six!"
    Case Else
        Print Value
End Select
```

Graphics recipe: one complete frame

Draw to the back buffer, then present once.

Minimal graphical program

```
Game Window "My First Game" Size 960 By 540

PlayerX = 480
PlayerY = 270
Key = KEY_NONE

Do
    Get Key Key

    If Key_Held(KEY_LEFT) = True Then
        PlayerX = Max(20, PlayerX - 5)
    End If
    If Key_Held(KEY_RIGHT) = True Then
        PlayerX = Min(940, PlayerX + 5)
    End If

    Clear Rgb(5, 8, 18)
    Fill Circle PlayerX, PlayerY, 12, CYAN
    Draw Text "Move With Arrows" At 480, 40
        Size 22 Color WHITE Centered
    Draw Number PlayerX At 20, 500 Size 18 Color YELLOW
    Show Screen
Loop Until Game_Closed() = True

End Program
```

Primitive shapes

- Rectangle and Rounded Rectangle
- Circle and Arc
- Quadrilateral for ships and perspective
- Line for borders, stars and effects

Text and numbers

Draw Text accepts a literal. Draw Number accepts a numeric expression. Both use At, Size and Color; Text can add Centered.

Presentation discipline

Avoid Show Screen after every individual shape. Build the whole frame first. One presentation produces a coherent image and matches double-buffering.

Logical coordinates

The default games use 960 by 540. The runtime scales that canvas to the physical window or fullscreen display while preserving aspect ratio.

Audio, storage and file-loading recipes

Data that lives outside the immediate frame.

Persistent integer

```
Load HighScore From "HighScore" Default 0

If Score > HighScore Then
    HighScore = Score
    Save HighScore To "HighScore"
End If
```

WAV effects and MP3 music

```
Play Sound "Assets\Start.wav"

Music Volume 38
Play Music "Assets\Background.mp3" Loop
Pause Music
Resume Music
Stop Music
```

Executable-relative UTF-8 file bytes

```
Const Capacity = 4096
Dim Bytes[Capacity]
Length = 0

Load Text File "Maps\default.map"
    Into Bytes Count Length

If Length = 0 Then
    Call BuildFallbackMap()
Else
    Call ParseLoadedMap()
End If
```

Design lesson

Loading bytes is intentionally lower-level than a magical map command. The student defines the file format, parses symbols, validates dimensions and reachability, and chooses a safe fallback. That turns an asset into a lesson about robust software boundaries.

Eight games, eight different programming lessons

Every game rule remains visible in SMILE source.

Two source editions

Each game folder contains Program.smile, the complete edition with attract/demo behavior, and Program-NoDemo.smile, the complete playable student edition without automated demo code. Both are real SMILE programs; the no-demo version makes the core mechanics easier to study.

How to read each three-page chapter

Page one shows title/gameplay reconstructions and the game’s purpose. Page two lists the unique language forms used by the core source and explains one important algorithm. Page three identifies what a student can learn and practical directions for extending the game.

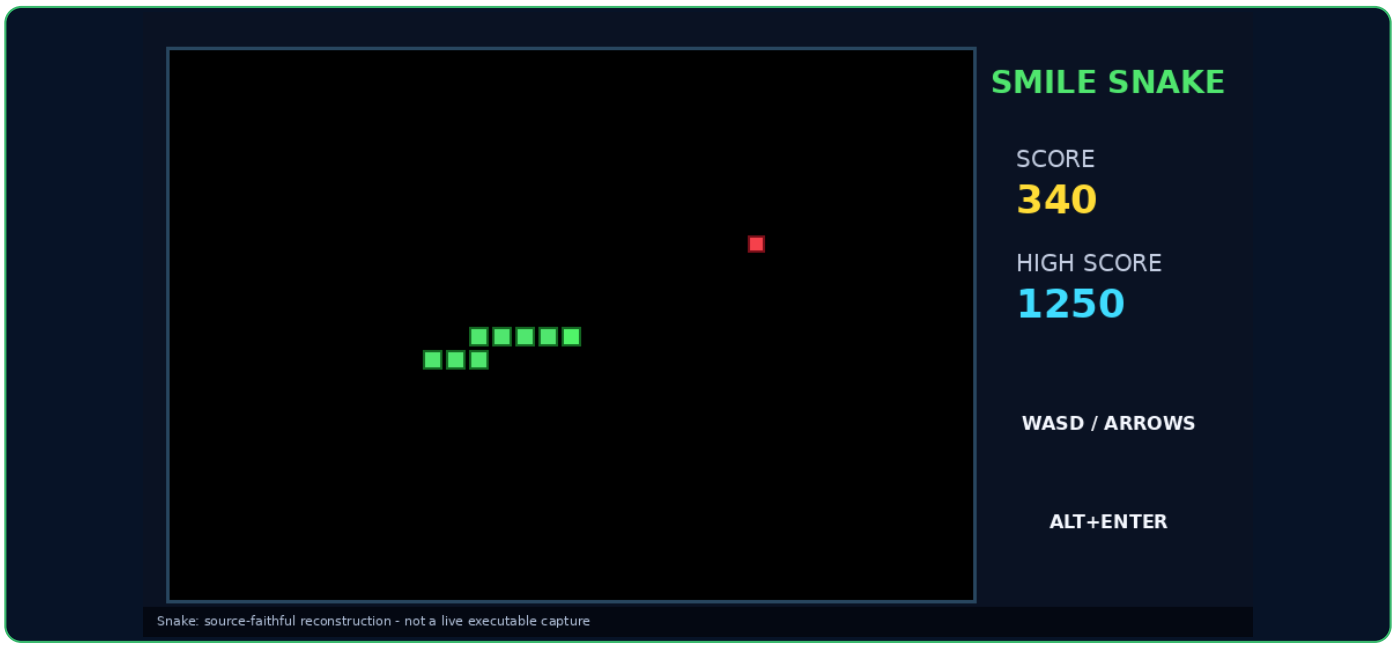
Game inventory verified from repository tree and each games/<name>/README.md at 92bf27e9d2

Snake

A grid-based arcade game in which the player steers a growing snake toward randomly placed food. The body...

A grid-based arcade game in which the player steers a growing snake toward randomly placed food. The body is stored as two fixed arrays of X and Y coordinates. Every movement shifts the body, advances the head, checks walls and self-collision, and either grows the snake or ends the round.

Title-screen reconstruction from committed drawing code

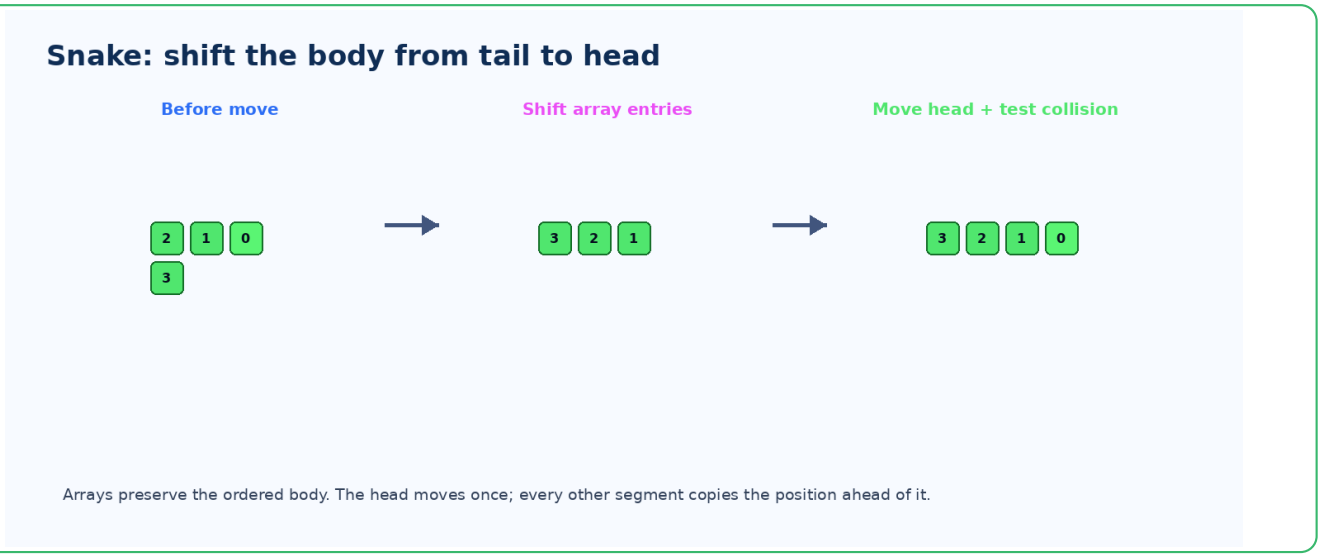


Gameplay reconstruction from committed drawing code

Sources: games/Snake/README.md; games/Snake/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

Snake: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.



Category	Commands / forms used
Data	Const, Dim, assignment, True, False
Flow	If / Else If / Else, For, Do / Loop Until
Routines	Sub, Call
Input + time	Get Key, Timer(), Random ... From ... To
Math + logic	Max, And, Or, comparisons
Graphics	Game Window, Clear, Fill/Draw Rectangle, Fill/Draw Rounded Rectangle, Draw Text, Draw Number, Show Screen
Audio + storage	Play/Stop Sound, Load ... Default, Save ... To
Program life	Game_Closed(), End Program

Representative source excerpt

```
Sub MoveSnake()
    Direction = NextDirection
    For I = Length - 1 Down To 1
        SnakeX[I] = SnakeX[I - 1]
        SnakeY[I] = SnakeY[I - 1]
    End For
    ... move head, test collision ...
End Sub
```

Snake: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

The important algorithm is the body shift. The loop copies each segment from the segment before it, working backward so old positions are not overwritten too early. Food generation repeats until it chooses a cell not occupied by the body.

What this game teaches

- Represent a moving object with ordered array elements.
- Use an update timer instead of tying speed to drawing rate.
- Reject illegal reverse-direction input.
- Generate random positions with a validation loop.
- Separate Reset, Update, Draw and EndRound responsibilities.
- Persist a high score between runs.

Ways a student could improve it

- Add wraparound walls as an optional mode.
- Add obstacles or level layouts.
- Add several food types with different points.
- Add pause and difficulty selection.
- Animate the head and tail differently.
- Create a map editor using a text file.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

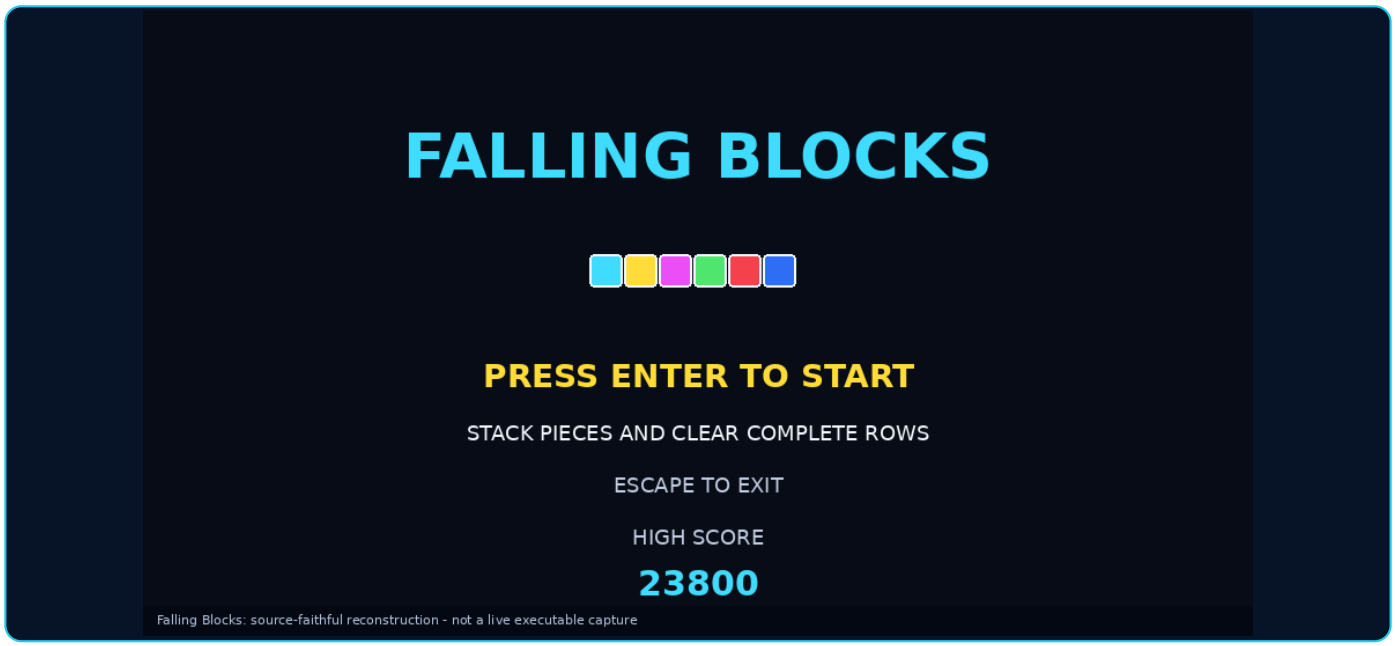
The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

Source paths: games/Snake/Program-NoDemo.smile; games/Snake/README.md; game project and assets at commit 92bf27e9d2

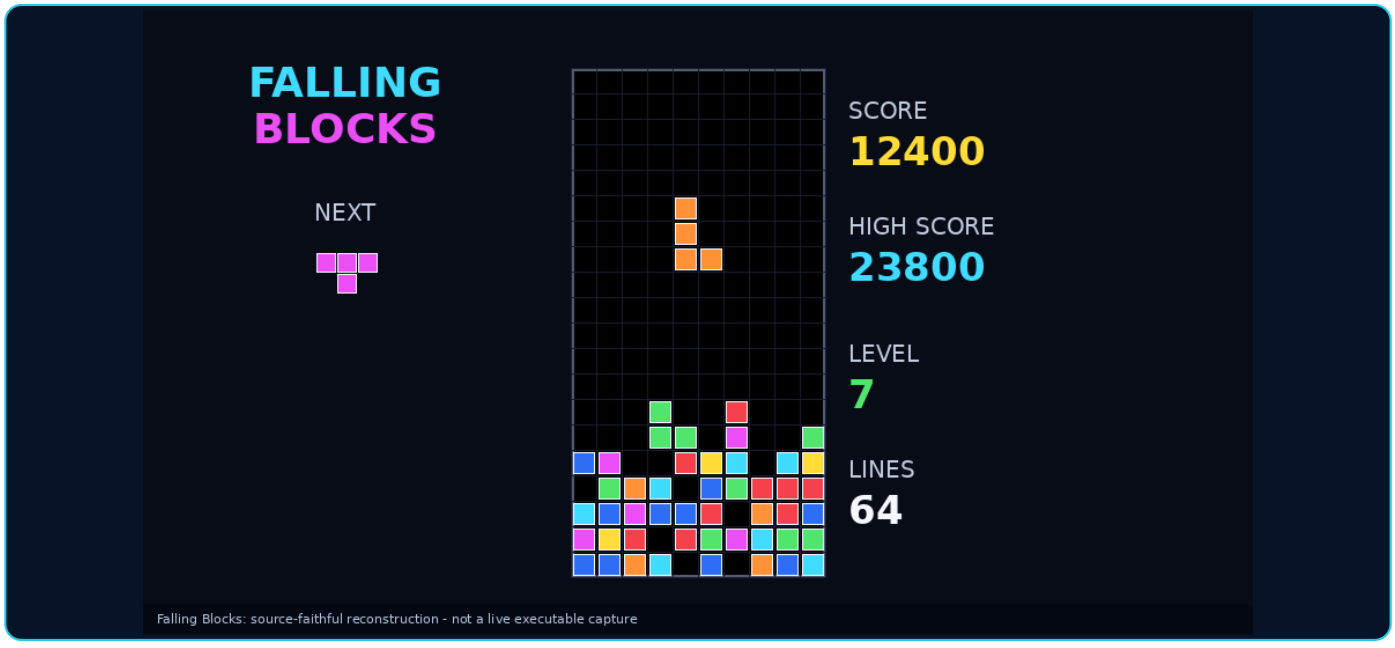
Falling Blocks

A 10-by-20 falling-piece puzzle. Seven four-cell shapes descend, rotate inside a 4-by-4 box, lock into th...

A 10-by-20 falling-piece puzzle. Seven four-cell shapes descend, rotate inside a 4-by-4 box, lock into the board, and clear complete rows. The source also demonstrates animated clear phases, level-based speed, score tables, a next-piece preview, looping MP3 music and WAV effects.



Title-screen reconstruction from committed drawing code



Gameplay reconstruction from committed drawing code

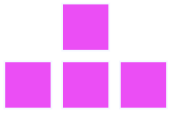
Sources: games/FallingBlocks/README.md; games/FallingBlocks/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

Falling Blocks: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.

Falling Blocks: rotate four cells, test, lock, clear, collapse

Rotation transform



`(x, y) -> (3 - y, x)`

Row-clear phases

1. Mark full rows
2. Flash / animate
3. Copy surviving rows downward
4. Zero the empty rows

Category	Commands / forms used
Data	Const, Dim, one- and two-dimensional arrays
Flow	If, For ascending/descending, Do/Loop, Exit Do, Select Case
Routines	Function/Return, Sub/Call
Random + time	Random, Timer, Min, Max, Mod
Input	Get Key and KEY_* constants
Graphics	Game Window, Clear, rectangles, text, numbers, Show Screen
Audio	Play/Stop Sound, Play/Stop Music, Loop
Storage	Load/Save high score; End Program

Representative source excerpt

```
Function BlockX(PieceValue, RotationValue, CellValue)
    CalcX = ShapeX[PieceValue - 1, CellValue]
    CalcY = ShapeY[PieceValue - 1, CellValue]
    For Turn = 1 To RotationValue
        RotatedX = 3 - CalcY
        CalcY = CalcX
        CalcX = RotatedX
    End For
    Return CalcX
End Function
```

Falling Blocks: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

Rotation repeatedly applies the integer coordinate transform (x, y) -> (3 - y, x). CanMove tests all four cells before a translation or rotation is accepted. Row clearing marks full rows first, animates them, copies surviving rows downward, then zeroes the empty top rows.

What this game teaches

- Model a board as a 2D array.
- Transform local shape coordinates.
- Validate a move before changing state.
- Use Select Case for piece colors and score tables.
- Implement multi-phase animation with Timer deadlines.
- Make difficulty increase from accumulated progress.

Ways a student could improve it

- Use a seven-bag randomizer.
- Add a ghost piece and hold slot.
- Implement wall kicks near edges.
- Add combo and back-to-back bonuses.
- Load level themes from files.
- Add configurable controls and music volume.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

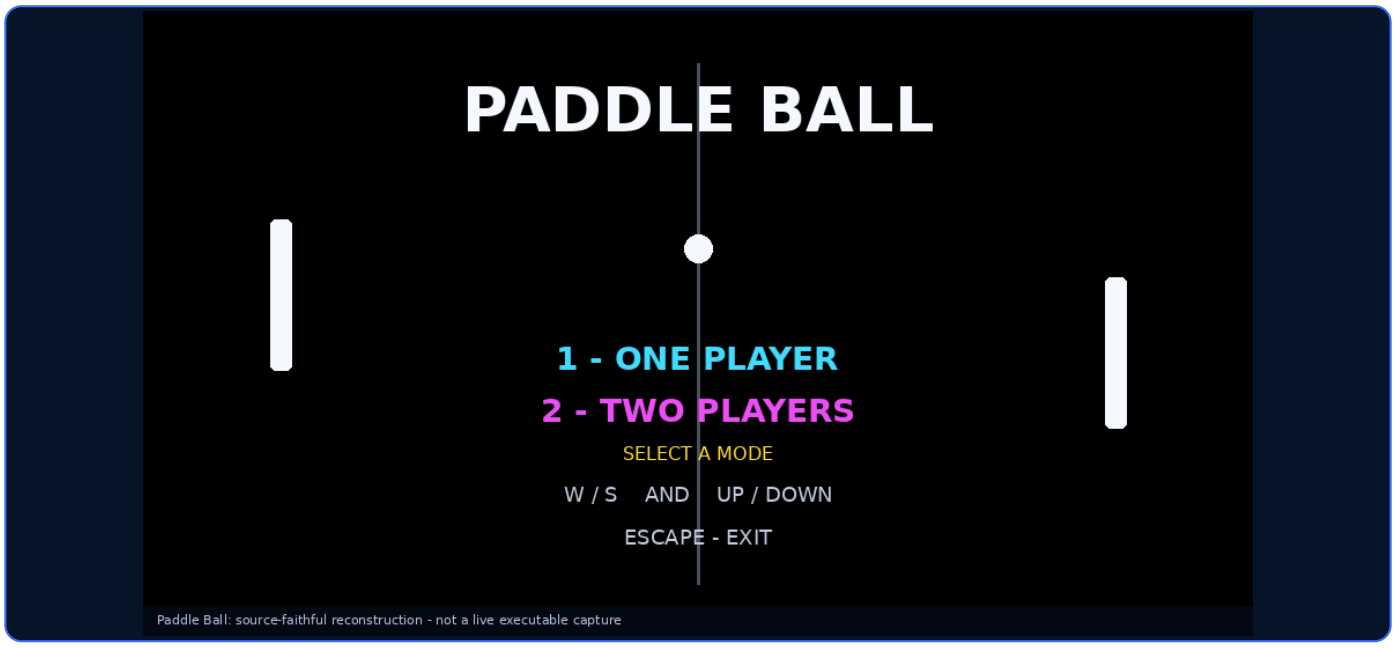
The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

Source paths: games/FallingBlocks/Program-NoDemo.smile; games/FallingBlocks/README.md; game project and assets at commit 92bf27e9d2

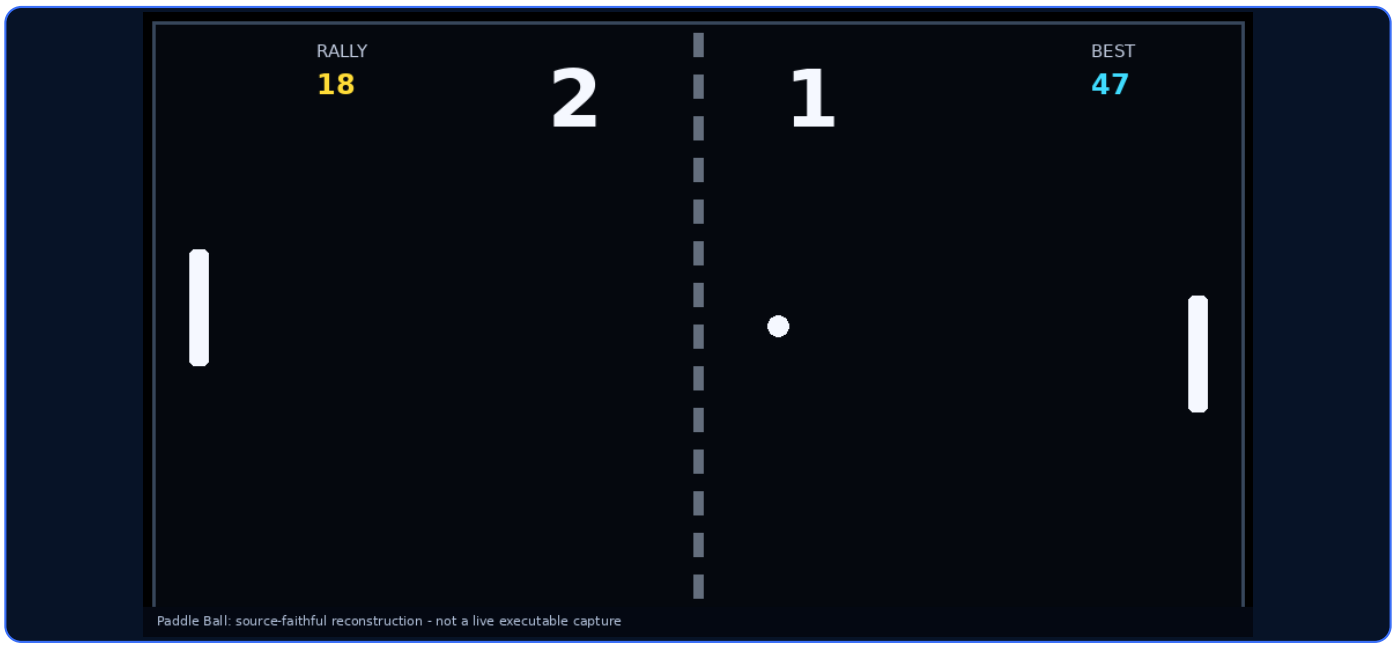
Paddle Ball

A one- or two-player paddle game. The one-player mode includes a deliberately beatable computer opponent,...

A one- or two-player paddle game. The one-player mode includes a deliberately beatable computer opponent, rally tracking and a persistent best rally. Movement uses 1,000 fixed-point subpixel units per pixel and an 8 ms fixed simulation step so speed remains stable on different monitors.



Title-screen reconstruction from committed drawing code

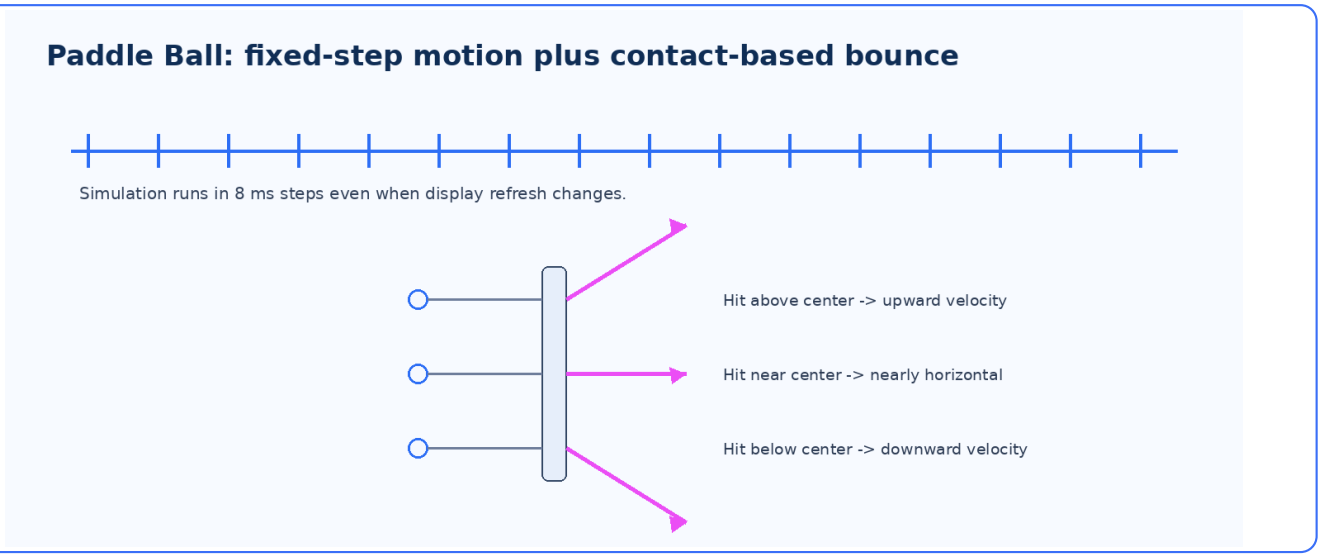


Gameplay reconstruction from committed drawing code

Sources: games/PaddleBall/README.md; games/PaddleBall/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

Paddle Ball: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.



Category	Commands / forms used
Data	Const and scalar game state
Flow	If/Else If/Else, For, Do/Loop Until
Routines	Sub/Call
Input	Get Key, Key_Held, KEY_1, KEY_2
Math + time	Timer, Random, Abs, Min, Max
Graphics	Game Window, Clear, rectangles, rounded rectangles, circle, line, text, numbers, Show Screen
Audio + storage	Play/Stop Sound, Load/Save BestRally
Program life	Game_Closed(), End Program

Representative source excerpt

```
Accumulator = Min(SimulationStep * MaxCatchUpSteps,
                  Accumulator + Elapsed)
Do
    If Accumulator >= SimulationStep Then
        Call UpdatePaddles()
        Call UpdateBall()
        Accumulator = Accumulator - SimulationStep
    End If
Loop Until Accumulator < SimulationStep
```

Paddle Ball: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

The bounce angle depends on where the ball touches the paddle. Contact above the center produces upward velocity; contact below produces downward velocity. Horizontal speed rises with rally length. An accumulator runs enough 8 ms updates to catch up while clamping long pauses.

What this game teaches

- Separate simulation speed from display refresh rate.
- Store fractions with fixed-point integers.
- Create simple but understandable computer control.
- Turn collision position into gameplay behavior.
- Support multiple modes with one state variable.
- Persist a performance record.

Ways a student could improve it

- Add paddle motion “spin” to the bounce.
- Offer easy, normal and hard AI speeds.
- Add best-of-three match rules.
- Add power-ups or moving obstacles.
- Add controller support.
- Add particles and optional background music.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

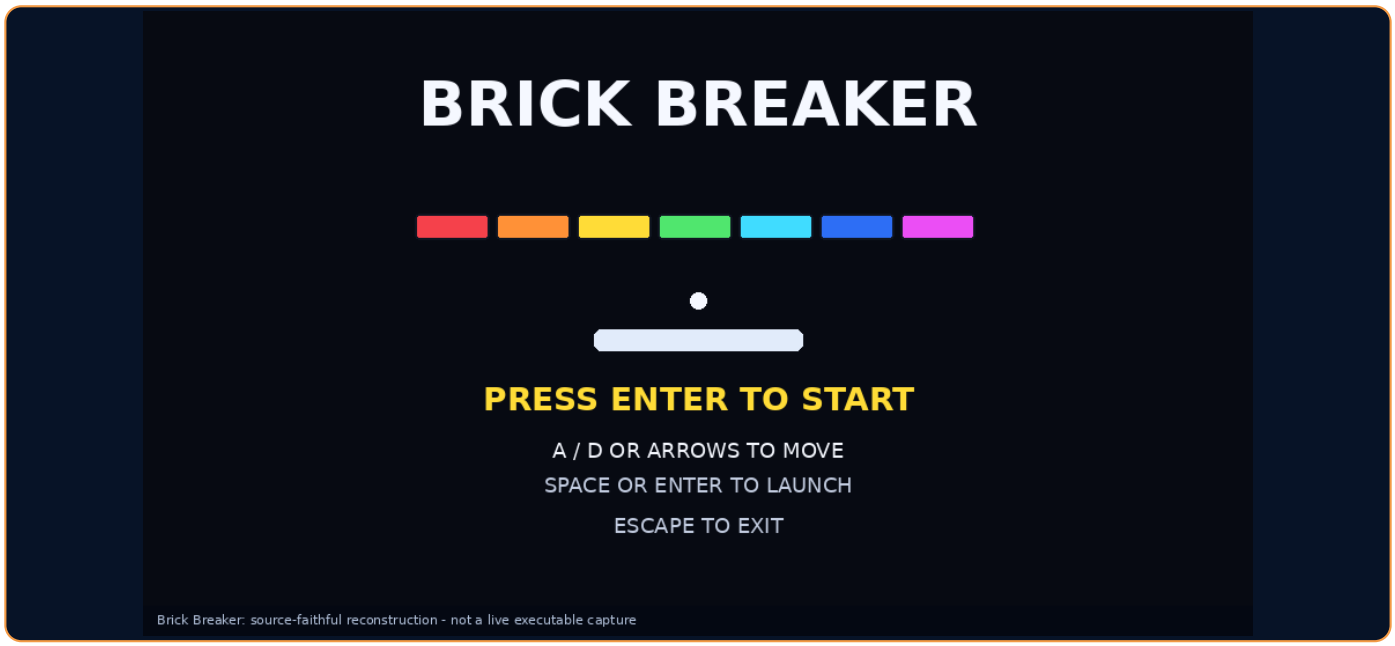
The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

Source paths: games/PaddleBall/Program-NoDemo.smile; games/PaddleBall/README.md; game project and assets at commit 92bf27e9d2

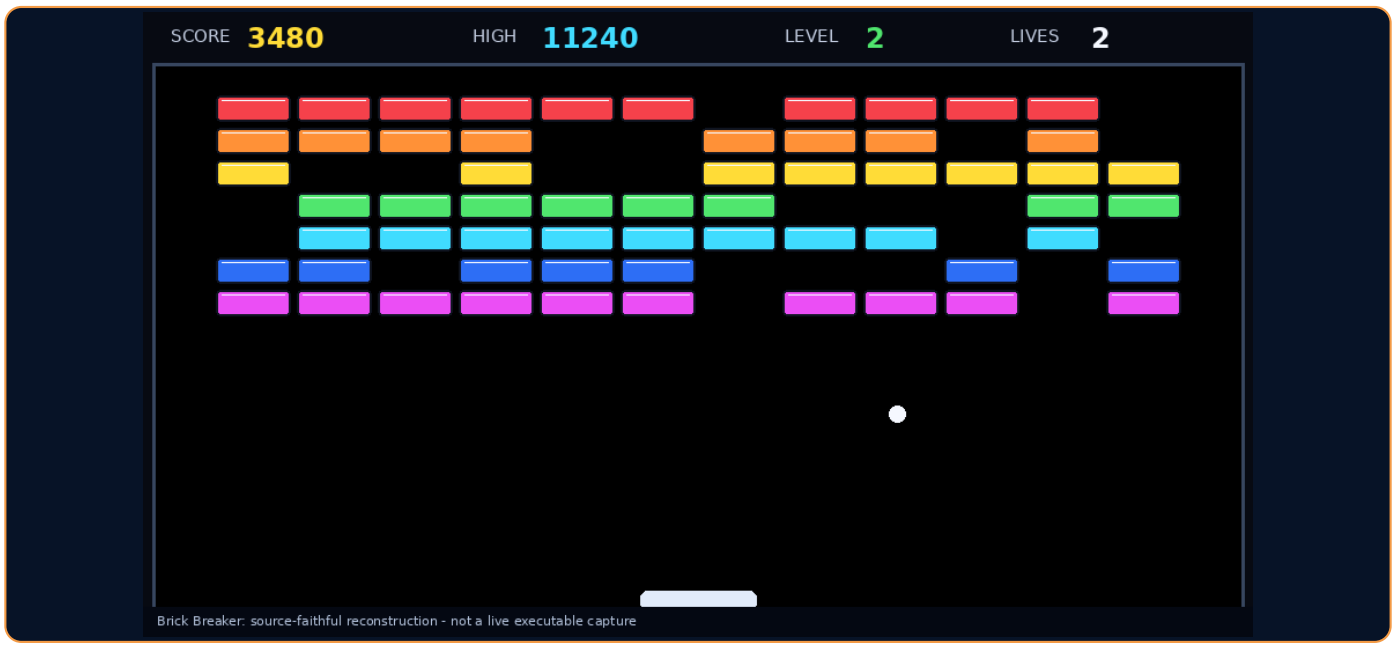
Brick Breaker

A three-level paddle-and-ball game with 84 bricks, three lives, score, high score and victory/game-over s...

A three-level paddle-and-ball game with 84 bricks, three lives, score, high score and victory/game-over states. Like Paddle Ball, it uses fixed-point movement and an 8 ms fixed step. Each brick row has a different color and point value.



Title-screen reconstruction from committed drawing code

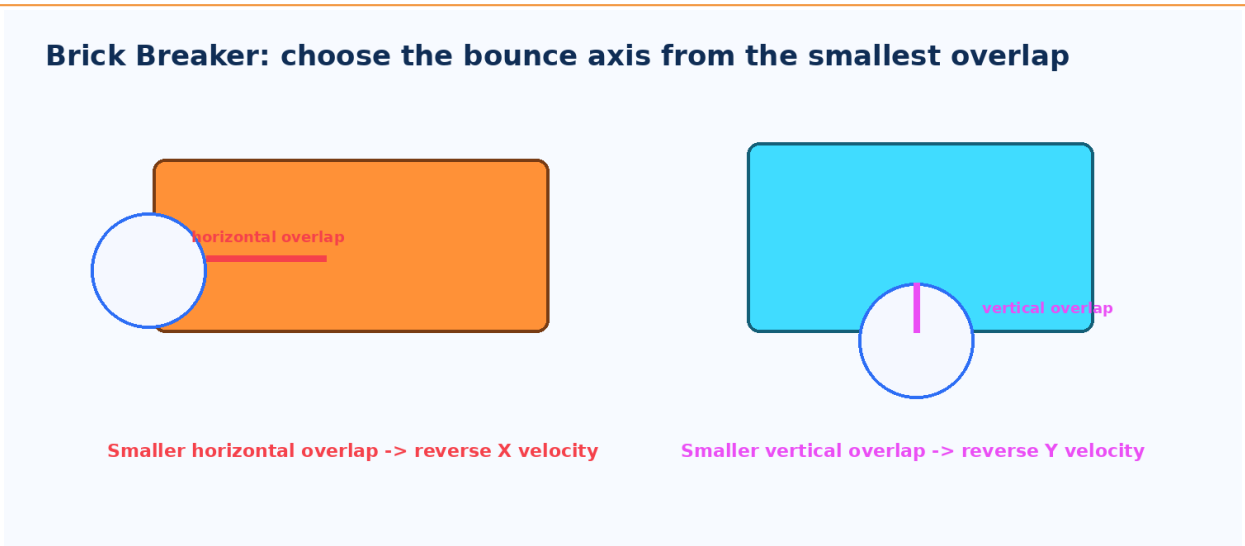


Gameplay reconstruction from committed drawing code

Sources: games/BrickBreaker/README.md; games/BrickBreaker/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

Brick Breaker: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.



Category	Commands / forms used
Data	Const, Dim Bricks[columns, rows]
Flow	If/Else If/Else, nested For, Do/Loop Until, Select Case
Routines	Function/Return, Sub/Call
Input	Get Key, Key_Held
Math + time	Timer, Abs, Min, Max
Graphics	Game Window, Clear, rectangles, rounded rectangles, circle, line, text, number, Show Screen
Audio + storage	Play Sound, Load/Save HighScore
Program life	Game_Closed(), End Program

Representative source excerpt

```
HorizontalOverlap = Min(OverlapLeft, OverlapRight)
VerticalOverlap = Min(OverlapTop, OverlapBottom)
If HorizontalOverlap < VerticalOverlap Then
    BallVX = -BallVX
Else
    BallVY = -BallVY
End If
```

Brick Breaker: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

When the ball overlaps a brick, the game measures overlap from the left, right, top and bottom. It reverses horizontal velocity when horizontal overlap is smaller; otherwise it reverses vertical velocity. This produces a more believable bounce than always reversing the same axis.

What this game teaches

- Store a field of destructible objects in a 2D array.
- Calculate simple rectangle/circle overlap responses.
- Use functions for row colors and point values.
- Build lives, levels, victory and game-over states.
- Reuse fixed-step timing from another genre.
- Keep drawing and collision data in logical coordinates.

Ways a student could improve it

- Give some bricks multiple hit points.
- Load level patterns from text files.
- Add falling power-ups.
- Make paddle movement influence bounce.
- Add particles and screen shake.
- Add music and a level-select menu.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

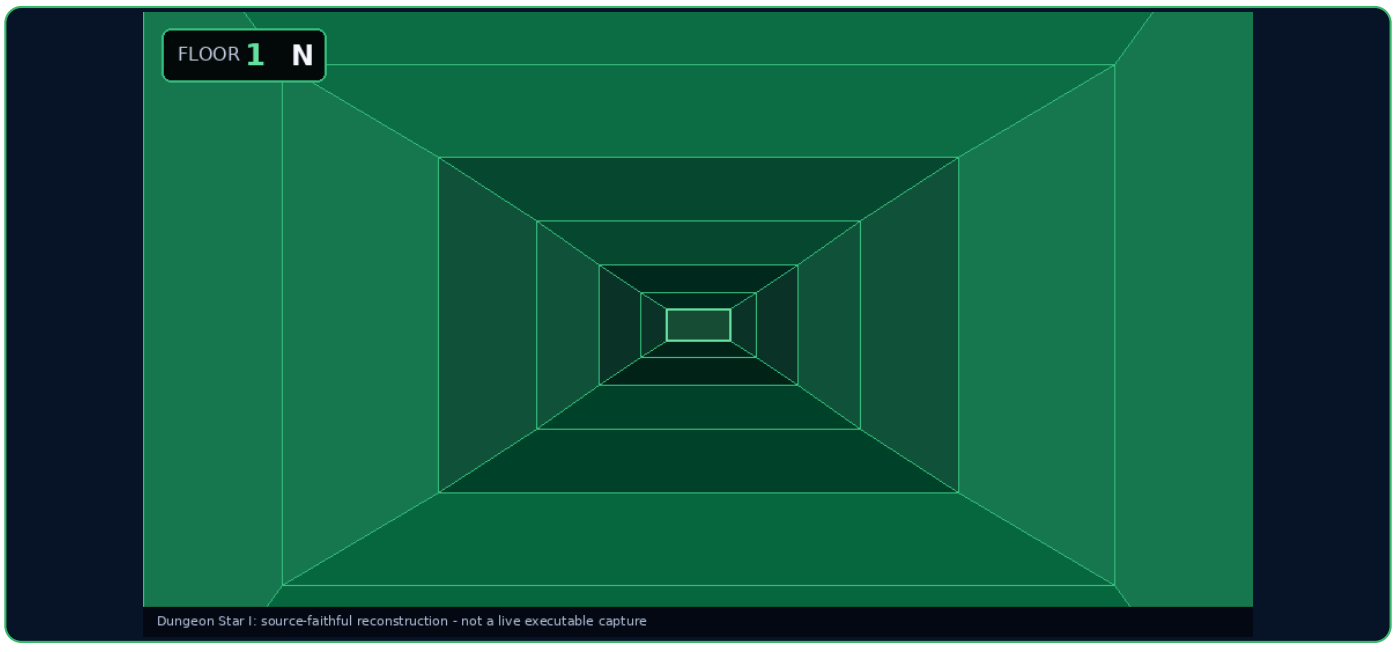
Source paths: games/BrickBreaker/Program-NoDemo.smile; games/BrickBreaker/README.md; game project and assets at commit 92bf27e9d2

Dungeon Star I

An original first-person, grid-based dungeon explorer with three 31-by-31 floors. It supports editable ex...

An original first-person, grid-based dungeon explorer with three 31-by-31 floors. It supports editable external maps, a bounded random pipe-style generator, closed/open doors, reciprocal stairs, palette-swapped pseudo-3D rendering, timed transitions, idle handling and looping MP3 music.

Title-screen reconstruction from committed drawing code



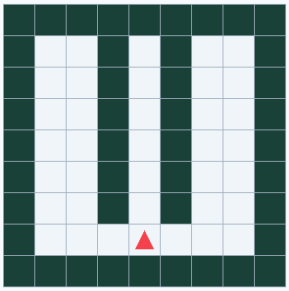
Gameplay reconstruction from committed drawing code

Sources: games/DungeonStarI/README.md; games/DungeonStarI/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

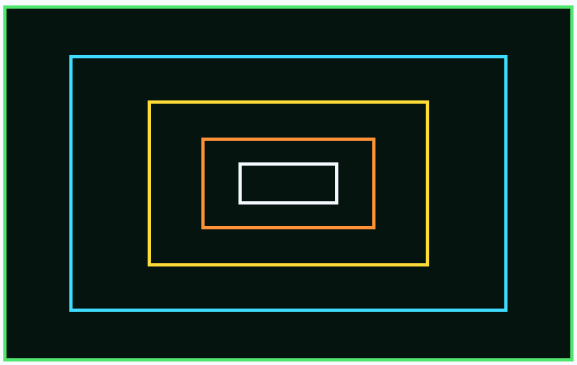
Dungeon Star I: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.

Dungeon Star I: a 2D grid becomes a pseudo-3D corridor



Grid cells answer: wall, floor, door or stairs.



Precomputed shrinking frames create depth without raycasting.

Category	Commands / forms used
Data	Many Const values and fixed Dim arrays for map, metadata, BFS and projection
Flow	If, nested For, Do/Loop Until, descending For, Return
Routines	Function/Return and Sub/Call
Files + generation	Load Text File, Random, validation and fallback
Input + time	Get Key, Key_Held, Timer, Mod, Min, Max
Graphics	Game Window, Clear, rectangles, rounded rectangles, quadrilaterals, lines, text, number, Show Screen
Audio	Play Music ... Loop, Stop Music
Program life	Game_Closed(), End Program

Representative source excerpt

```
ActionProgress = Min(1000,
    (Now - ActionStartedAt) * 1000 / TurnDuration)
If ActionProgress >= 1000 Then
    PlayerDirection = TargetDirection
    Call EnterIdle()
End If
```

Dungeon Star I: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

The world is still a 2D array. The renderer looks forward through a small number of cells and draws nested quadrilaterals whose widths and heights shrink with depth. Movement, turning, door opening and stairs are explicit action states with progress from 0 to 1000.

What this game teaches

- Separate a world map from its first-person rendering.
- Validate external data before trusting it.
- Use flood fill/BFS to prove connectivity.
- Build a bounded procedural generator with deterministic fallback.
- Animate actions with a state machine and normalized progress.
- Keep all game rules in editable SMILE source.

Ways a student could improve it

- Add an optional minimap.
- Add keys, locks and secret doors.
- Add treasure or simple inventory.
- Create a graphical map editor.
- Add decorative geometry and lighting variations.
- Add simple encounters while preserving beginner readability.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

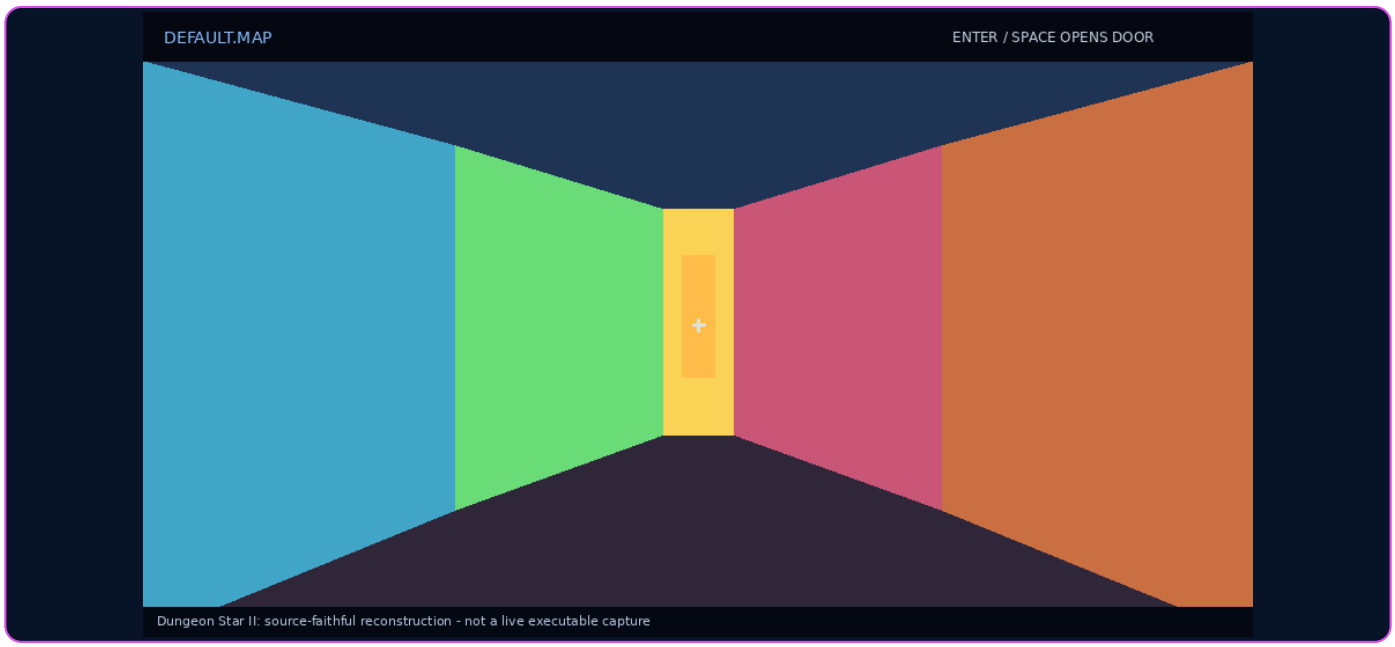
Source paths: games/DungeonStarI/Program-NoDemo.smile; games/DungeonStarI/README.md; game project and assets at commit 92bf27e9d2

Dungeon Star II

A fixed-point raycasting walkaround. A one-floor 31-by-31 map can contain rooms, corridors, loops, colore...

A fixed-point raycasting walkaround. A one-floor 31-by-31 map can contain rooms, corridors, loops, colored wall materials, pillars and rising doors. The 960-by-540 view casts 960 rays, records the first hit, and merges consecutive hits on one wall plane into anti-aliased quadrilaterals.

Title-screen reconstruction from committed drawing code

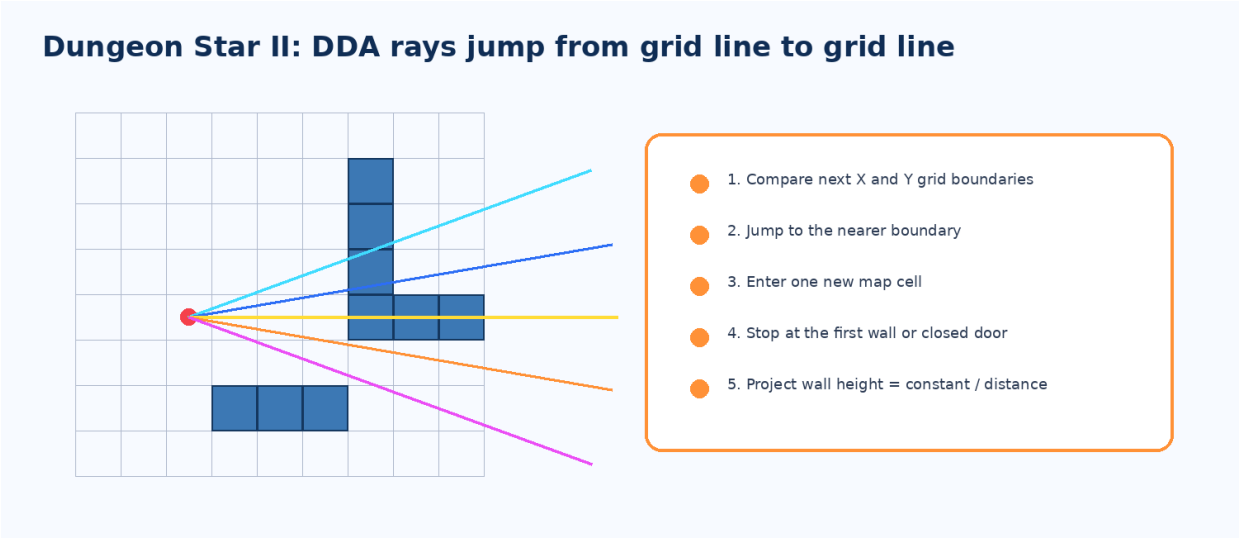


Gameplay reconstruction from committed drawing code

Sources: games/DungeonStarII/README.md; games/DungeonStarII/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

Dungeon Star II: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.



Category	Commands / forms used
Data	Const scaling values; Dim maps, ray buffers, rooms, queues and palettes
Flow	If/Else If, nested For, Do/Loop Until
Routines	Function/Return, Sub/Call
Files + generation	Load Text File, Random rooms/corridors, validation fallback
Math + time	Timer, Abs, Min, Max, Mod; fixed-point integer math
Input	Get Key, Key_Held
Graphics	Game Window Size ... By ..., Clear, rectangles, rounded rectangles, quadrilaterals, text, Show Screen
Program life	Game_Closed(), End Program

Representative source excerpt

```
If SideDistanceX < SideDistanceY Then
    SideDistanceX = SideDistanceX + DeltaDistanceX
    RayMapX = RayMapX + StepX
    RayHitSide = 0
Else
    SideDistanceY = SideDistanceY + DeltaDistanceY
    RayMapY = RayMapY + StepY
    RayHitSide = 1
End If
```

Dungeon Star II: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

DDA compares the distance to the next vertical and horizontal grid boundary, jumps to the nearer one, and enters exactly one new cell. Perpendicular forward distance determines wall height and avoids fish-eye curvature. Player X and Y collision are tested separately so the player can slide along walls.

What this game teaches

- Understand how a 2D map can make a 2.5D view.
- Use scaled integers when floating point is unavailable.
- Implement DDA grid traversal.
- Use projection and perpendicular distance.
- Interpolate rendering between fixed simulation steps.
- Merge samples into longer anti-aliased geometry.

Ways a student could improve it

- Add wall textures.
- Add billboard sprites and a depth buffer.
- Add doors that close again.
- Add positional sound and music.
- Add animated enemies and one simple weapon.
- Experiment with field of view and ray count.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

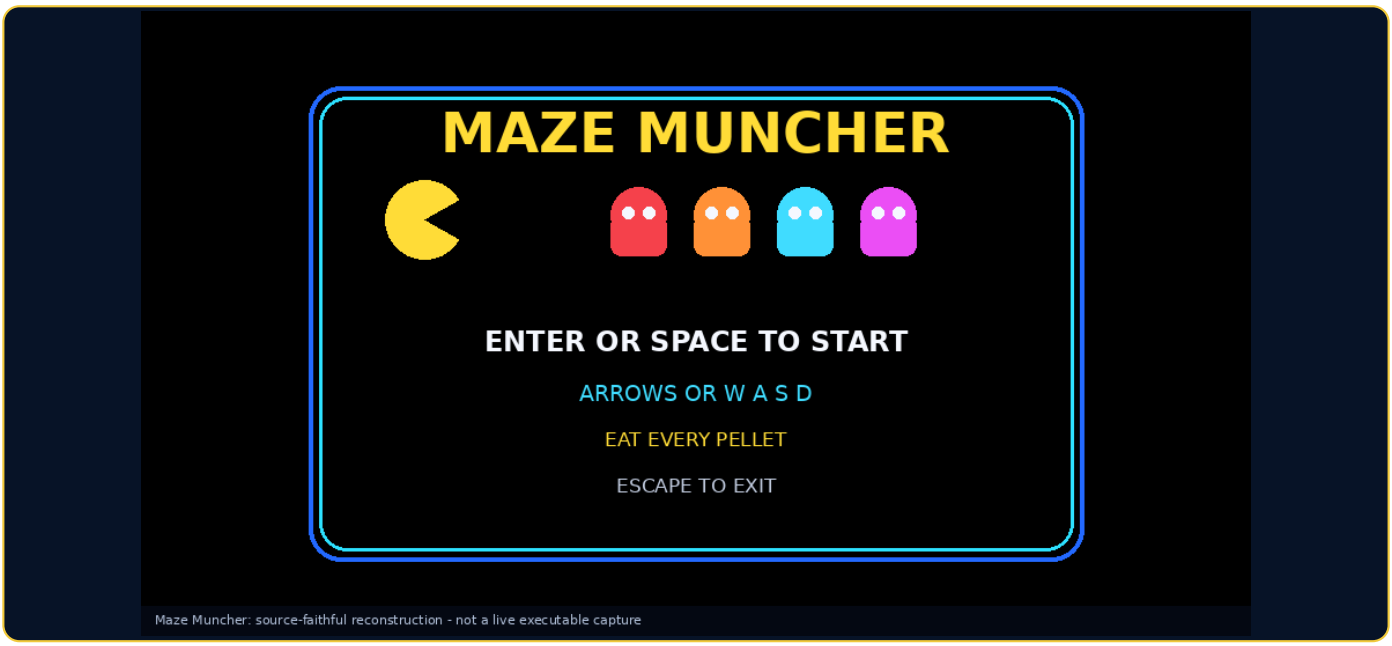
The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

Source paths: games/DungeonStarII/Program-NoDemo.smile; games/DungeonStarII/README.md; game project and assets at commit 92bf27e9d2

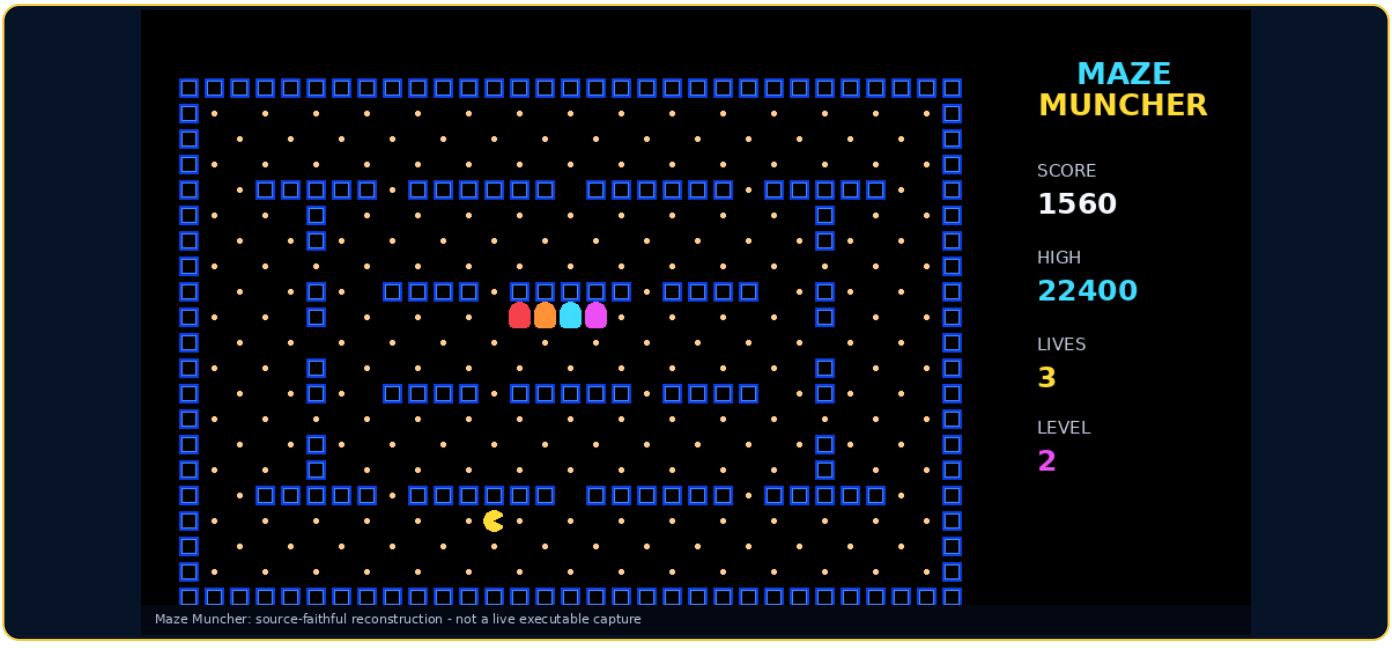
Maze Muncher

An original neon maze-chase game. The player eats normal and power pellets, uses a horizontal wrap tunnel...

An original neon maze-chase game. The player eats normal and power pellets, uses a horizontal wrap tunnel, avoids four geometric enemies, advances through levels and stores a high score. An external 31-by-21 map is parsed and validated; missing or invalid data uses a deterministic safe maze.



Title-screen reconstruction from committed drawing code

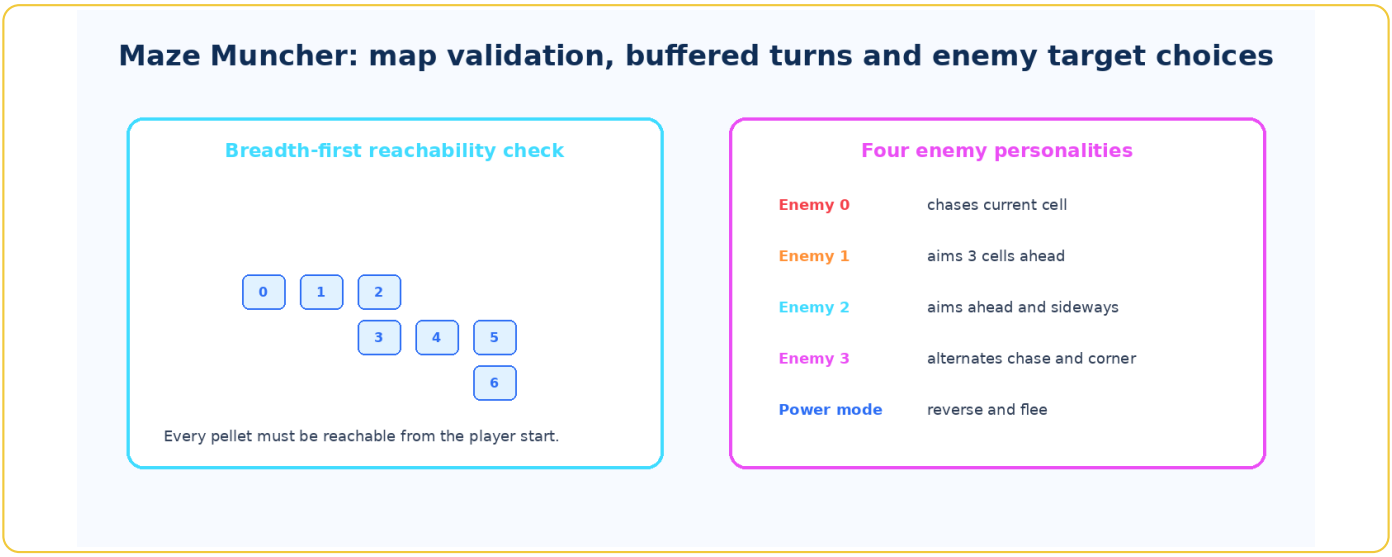


Gameplay reconstruction from committed drawing code

Sources: games/MazeMuncher/README.md; games/MazeMuncher/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

Maze Muncher: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.



Category	Commands / forms used
Data	Const and many one-/two-dimensional Dim arrays
Flow	If/Else If, For, Do/Loop Until, Return
Routines	Function/Return, Sub/Call
Files + validation	Load Text File, BFS queue/visited arrays, deterministic fallback
Math + time	Timer, Abs, Min, Max, Mod; fixed-step movement
Graphics	Clear, lines, Draw Arc, circles, rounded rectangles, quadrilaterals, text, numbers, Show Screen
Audio	Play/Stop Sound, Play/Stop Music, Music Volume
Storage	Load/Save HighScore; Game_Closed; End Program

Representative source excerpt

```
CandidateScore =
    (Abs(CandidateX - TargetX) +
     Abs(CandidateY - TargetY)) * 10
CandidateScore = CandidateScore +
    Min(60, CandidateVisit * 7)
```

Maze Muncher: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

The parser reads UTF-8 bytes and recognizes wall, pellet, power, gate, tunnel, player and enemy symbols. BFS proves that every pellet is reachable. Buffered input remembers the next requested turn. Each enemy evaluates legal directions against a different target and a visit penalty; power mode reverses behavior.

What this game teaches

- Parse a human-editable map format.
- Validate required symbols and reachability.
- Use buffered direction for responsive grid movement.
- Build several enemy personalities from one scoring function.
- Use timers for power mode and staged release.
- Draw complex neon contours from simple primitives.

Ways a student could improve it

- Add several selectable maps.
- Add bonus fruit or timed objectives.
- Create scatter/chase mode schedules.
- Add animated death and level transitions.
- Make a visual maze editor.
- Add per-level palettes, speed tables and music.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

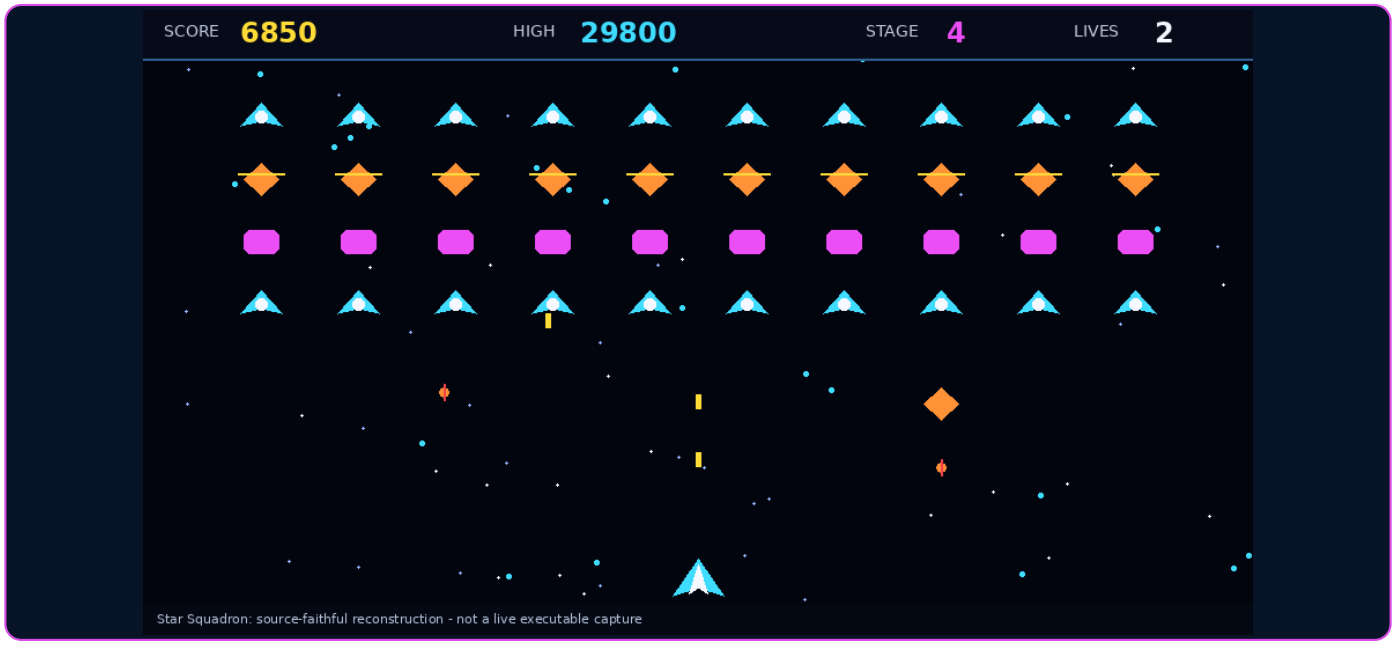
Source paths: games/MazeMuncher/Program-NoDemo.smile; games/MazeMuncher/README.md; game project and assets at commit 92bf27e9d2

Star Squadron

A full-width fixed-screen space shooter. A 72-star multi-speed field runs behind a 40-enemy formation. En...

A full-width fixed-screen space shooter. A 72-star multi-speed field runs behind a 40-enemy formation. Enemies drift, fire, dive through piecewise paths and return to formation. The player has three lives, three simultaneous shots, stage progression, geometric explosions, score and persistent high score.

Title-screen reconstruction from committed drawing code



Gameplay reconstruction from committed drawing code

Sources: games/StarSquadron/README.md; games/StarSquadron/Program-NoDemo.smile; Program.smile - snapshot 92bf27e9d2

Star Squadron: commands and core algorithm

Unique language forms in the complete no-demo teaching source; Program.smile adds attract-mode AI with the same public language surface.

Category	Commands / forms used
Data	Const plus Dim arrays for stars, enemies and projectile pools
Flow	If/Else If, For, Do/Loop Until, Return
Routines	Function/Return, Sub/Call
Random + time	Random, Timer, Abs, Min, Max, Mod
Input	Get Key, Key_Held
Graphics	Clear, rectangles, rounded rectangles, circles, quadrilaterals, lines, text, numbers, Show Screen
Audio + storage	Play/Stop Sound, Load/Save HighScore
Program life	Game Window, Game_Closed(), End Program

Representative source excerpt

```
If EnemyState[EnemyIndex] = ENEMY_DIVING Then
    EnemyDiveProgress[EnemyIndex] =
        EnemyDiveProgress[EnemyIndex] + SimulationStep
    ... follow one of several path phases ...
Else If EnemyState[EnemyIndex] = ENEMY_RETURNING Then
    ... move toward formation home ...
End If
```

Star Squadron: what the student learns

A playable project is a collection of small, teachable ideas.

The distinctive mechanic

Fixed arrays act as object pools for enemies, player shots and enemy shots. Each enemy has an alive flag, type, state, position, hit points and dive progress. A state value moves an enemy from formation to diving to returning. Stage number shortens attack delays and increases shot speed.

What this game teaches

- Use parallel arrays as a simple entity system.
- Reuse fixed slots instead of allocating objects during play.
- Build behavior from explicit enemy states.
- Create escalating stages with timing formulas.
- Test many projectile/enemy collisions.
- Draw recognizable art entirely from geometry.

Ways a student could improve it

- Add enemy formations loaded from data.
- Add bosses and power-ups.
- Add scrolling stages.
- Add music and layered sound volume.
- Add controller support and remapping.
- Add replay recording or a score table.

How to study the source

Start with the constants and state variables. Then find the routine that resets a round, the routine that updates one simulation step, and the routine that draws the screen. Finally trace one input from Get Key or Key_Held through state change to visible output.

The demo-enabled Program.smile is best read afterward. It adds autonomous decisions and arcade lifecycle behavior without replacing the real game rules.

Source paths: games/StarSquadron/Program-NoDemo.smile; games/StarSquadron/README.md; game project and assets at commit 92bf27e9d2

Technical jargon translated - 1 of 6

Correct terms, explained without assuming prior compiler or game-engine experience.

Term	Plain-English explanation
Syntax	The spelling and structure rules of a language. In SMILE, End If is valid syntax; Endif is not.
Semantics	The meaning of syntactically valid code. Semantic checks ask whether names exist, arguments match, and values are used sensibly.
Keyword	A reserved word with a language-defined job, such as If, Dim or Draw. It cannot be reused as an ordinary variable name.
Literal	A value written directly in source code, such as 42, True or "Hello".
Variable	A named storage location whose value can change while the program runs.
Constant	A named value declared with Const that cannot be reassigned. It makes important numbers readable.
Expression	Code that calculates a value, such as Score + 10 or X < Width.
Statement	A complete instruction, such as Print Score or Call ResetGame().
Operator	A symbol or word that combines values: +, -, *, /, Mod, And, Or and comparison operators.
Scope	The region where a name is visible. Routine parameters and local temporaries belong to a routine; top-level game state is shared.
Lexer / tokenizer	The first language-analysis stage. It turns characters into labeled tokens such as If, identifier, number and plus sign.
Token	One recognized piece of source code together with its kind, position and optional value.
Parser	The stage that checks grammatical order and builds structured syntax nodes from tokens.

Technical jargon translated - 2 of 6

Correct terms, explained without assuming prior compiler or game-engine experience.

Term	Plain-English explanation
Grammar	The formal rules that describe which token sequences form valid statements and expressions.
Syntax tree / AST	A tree-shaped representation of code. Parent nodes describe larger ideas; child nodes hold their parts.
Symbol	The compiler’s record for a declared name such as a variable, array, constant, Sub or Function.
Type checking	Verifying that an operation receives the kind of value it expects. SMILE currently centers on signed 64-bit integers and Booleans.
Diagnostic	A precise error or warning with an ID, message and source span. Visual Studio and smilec share the same diagnostics.
Code generation / emitter	The stage that translates the checked program into lower-level MASM x64 assembly text.
Assembly language	A readable form of CPU-level instructions and calls. SMILE uses Microsoft Macro Assembler syntax.
MASM	Microsoft Macro Assembler. ml64.exe turns generated x64 assembly into an object file.
x64	The 64-bit instruction set used by modern AMD and Intel Windows PCs.
Object file	Machine code and metadata that are not yet a complete program. The linker combines object files and libraries.
Linker	A tool that resolves references between compiled pieces and produces the final executable.
Library	Reusable compiled code. Generated games link against the SMILE native runtime library and Windows libraries.
Native executable	A .exe containing machine code for the operating system and processor, rather than managed CLR bytecode.

Technical jargon translated - 3 of 6

Correct terms, explained without assuming prior compiler or game-engine experience.

Term	Plain-English explanation
Entry point	The first function the operating system calls when the program starts. SMILE’s toolchain controls this explicitly.
ABI	Application Binary Interface: the low-level rules for calls, registers, stack layout and binary compatibility.
C ABI	A stable C-style function boundary used so generated assembly can call runtime code implemented in C or C++.
Runtime	Generic support code used while a generated program runs: windows, input, graphics, audio, files, time and storage.
Win32	The classic Windows desktop API family used for windows, messages, file paths, input and other operating-system services.
DirectX	Microsoft’s family of graphics and media APIs. SMILE’s DirectX backend combines D3D11, Direct2D and DirectWrite.
Direct3D 11 / D3D11	Creates the graphics device and swap chain that present DirectX frames to the window.
Direct2D	Draws anti-aliased 2D geometry such as rectangles, circles, lines and quadrilaterals.
DirectWrite	Draws sharp text and measures centered labels in the DirectX backend.
GDI	Windows Graphics Device Interface. SMILE maintains it as a physical-resolution fallback backend.
Backend	One implementation hidden behind a common interface. The same SMILE Draw command can route to DirectX or GDI.
Back buffer	An off-screen image where a frame is built before it becomes visible.
Double buffering	Drawing away from the visible screen and presenting a finished frame at once, reducing flicker and tearing.

Technical jargon translated - 4 of 6

Correct terms, explained without assuming prior compiler or game-engine experience.

Term	Plain-English explanation
Swap chain	A small set of DirectX presentation buffers that alternate as frames are displayed.
VSync	Synchronizing presentation with the display refresh to reduce tearing.
Frame pacing	Controlling when frames are presented so motion appears regular instead of uneven.
DPI	Dots per inch, used by Windows display scaling. SMILE recalculates physical rendering while preserving logical coordinates.
Logical resolution	The coordinate system the game author uses, usually 960 by 540, regardless of physical window size.
Viewport	The physical rectangle where the logical canvas is scaled and centered.
Letterboxing	Black bars used when the physical aspect ratio does not match the game’s 16:9 canvas.
Fixed timestep	Updating game simulation in equal time slices, such as 8 ms, even when rendering occurs at a different rate.
Fixed-point math	Representing fractions with scaled integers. For example, 1 pixel may equal 1,000 subpixel units.
Subpixel	A fraction of a logical pixel stored as a scaled integer so motion can be smooth and refresh-independent.
State machine	A design where the program is in one named state at a time, such as TITLE, PLAYING or GAME_OVER.
Collision detection	Testing whether game objects or map regions touch or overlap.
AABB	Axis-aligned bounding box: a rectangle collision test that compares left, right, top and bottom edges.

Technical jargon translated - 5 of 6

Correct terms, explained without assuming prior compiler or game-engine experience.

Term	Plain-English explanation
Breadth-first search / BFS	A queue-based graph search that explores nearby cells first. It validates reachable maps and plans demo routes.
Queue	A first-in, first-out collection. BFS removes the oldest discovered cell and appends new neighbors.
DDA	Digital Differential Analyzer. Dungeon Star II jumps a ray directly between map-grid boundaries.
Raycasting	Sending imaginary rays through a 2D map and using wall distance to construct a first-person picture.
Camera plane	A sideways vector across the raycaster view. Each screen ray is direction plus part of this plane.
Projection	Turning distance into screen size. Near walls become tall; far walls become short.
Fish-eye correction	Using perpendicular forward distance so straight walls do not appear curved across the screen.
Pathfinding	Computing a route through legal map cells. Demo modes use it to play without user input.
VSIX	A Visual Studio extension package. SMILE’s VSIX supplies project support, language services and templates.
.smileproj	An XML project file naming the startup source, assets, graphics backend and VSync setting.
Asset	A file shipped beside the executable, such as WAV, MP3 or a text map.
Persistence	Saving a small value so it survives the process, such as a high score.

Technical jargon translated - 6 of 6

Correct terms, explained without assuming prior compiler or game-engine experience.

Term	Plain-English explanation
UTF-8	A standard text encoding. Load Text File exposes executable-relative UTF-8 bytes to SMILE code.
Asynchronous	Work that begins without blocking the game loop until completion. WAV and MP3 playback are asynchronous.
Deterministic	Given the same inputs and state, the algorithm behaves predictably. Deterministic fallbacks make failures safe to study.
Fallback	A safe alternate path, such as random map generation when an external map is missing or invalid.
Smoke test	A broad test that quickly checks whether major features build and run together.
Regression test	A test that preserves a previously correct behavior so a later change cannot silently break it.
Test harness	Automation that builds samples, runs checks and reports pass/fail consistently.
Single source of truth	One authoritative implementation. Smile.Language owns lexing, parsing and diagnostics for both CLI and Visual Studio.
Backend abstraction	A common interface hiding implementation details so DirectX and GDI can be swapped without changing SMILE syntax.
Vtable	A table of function pointers used by the native graphics router to call the selected backend.
COM	A Windows component model used by DirectX-related APIs and MediaPlayer infrastructure.
C++/WinRT	Microsoft’s C++ projection for Windows Runtime APIs, used by SMILE’s MP3 MediaPlayer module.

What SMILE 2.0 intentionally does not do yet

A clear boundary is healthier than pretending every feature already exists.

Current boundary	What it means today	A possible future learning milestone
Windows x64 only	Generated programs use Win32 and Microsoft native tools.	Introduce a platform abstraction and another native backend/toolchain.
Integer-centered values	No general floating-point type; games use fixed-point scaling.	Add Float after defining type conversions and code generation carefully.
Fixed arrays, max 2D	Sizes are compile-time constants; no dynamic list/collection.	Add user-defined records and safe dynamic collections.
Routines up to four scalar parameters	Enough for current samples, but large data is kept in shared arrays/state.	Generalize calling convention support and richer parameter types.
Literal-oriented text	Graphics text is literal; text files expose bytes rather than a full string library.	Add first-class strings, length/index operations and safe formatting.
Single WAV and MP3 channels	Simple and deterministic, but not a mixer.	Add multiple effect channels, fades and per-effect volume.
No textures/sprites yet	Games deliberately prove what geometry and text can accomplish.	Add image assets, sprite drawing and depth-aware billboards.
No package/module system	Programs are currently centered on one source file plus assets.	Design imports/modules only after naming and visibility rules are formalized.

A sensible learning order

The next feature should be chosen because it unlocks a clear student project, not because another language has it. A strong sequence is: formalize the feature, add lexer/parser/semantic support, emit native code, update the IDE, add tests, write a small example, then prove it inside a game.

Sources, scope and confidence

How to verify or update this guide.

Confidence level

High for version identity, architecture, language inventory, public syntax, runtime capabilities and game mechanics because they were checked against the exact committed source snapshot. Medium-high for the visual reconstructions because they follow drawing code and constants but were not live-captured from Windows executables.

Topic	Repository path(s)
Version and snapshot	src/Smile.VisualStudio/source.extension.vsixmanifest; Git commit 92bf27e9d2ae2b06baa486317e3346f80b5d3d83
Project overview	README.md
Architecture	docs/architecture/README.md
Language overview	docs/language/README.md
Keyword authority	src/Smile.Language/Syntax.cs; Lexer.cs; StructuredSyntax.cs; GameSyntax.cs
DirectX/GDI	docs/implementation/direct2d-implementation-report.md
MP3 MediaPlayer	docs/implementation/mp3-mediaplayer-implementation-report.md
Raycasting lesson	games/DungeonStarII/RAYCASTING_EXPLAINED.md
Games	games/<GameName>/README.md, Program.smile and Program-NoDemo.smile for all eight games
Build/validation	scripts/build.cmd; scripts/smoke-test.cmd; project test sources and manual checklists

Reproducibility note

Use repository Sincioco/SMILE-2.0 and check out commit 92bf27e9d2ae2b06baa486317e3346f80b5d3d83. The explicit packaged extension version in that source is 2.0.1. This PDF was generated at August 9, 2026 at 4:30:36 PM PHT (UTC+08:00).

The document deliberately does not claim live Windows screenshots. Every game image is marked as a source-faithful reconstruction. Rebuilding the projects on Windows and replacing those images with captured frames would be a valid future visual-only revision without changing the technical text.

End of Snapshot Guide

SMILE 2.0

Version 2.0.2 update

Multiline Parenthesized Expressions

An explicit, readable continuation rule for long expressions and call arguments. This update supplements the source snapshot in Version 2.0.1 and reflects Visual Studio extension version 2.0.33.

The rule in one sentence: a newline remains significant everywhere except inside balanced expression parentheses, where it acts as continuation whitespace.

Why parentheses are required

SMILE is still a line-oriented teaching language. Parentheses give the student and the parser the same visible signal that an expression continues. There is no underscore, backslash, semicolon, new keyword, or automatic continuation outside parentheses.

Preferred long If layout

Keep a complete **If ... Then** line on one line when it is at most 100 characters and has no more than two top-level Boolean clauses. Otherwise use one clause per line. The repository's preferred generated style keeps each logical operator at the end of its continued line:

```
If (Style.CursorWidth < 0 Or
    Style.CursorWidth > Core.UI_MAX_LAYOUT_VALUE Or
    Style.CursorHeight < 0 Or
    Style.CursorHeight > Core.UI_MAX_LAYOUT_VALUE) Then
    Return False
End If
```

The leading-operator layout is also legal source:

```
If (Style.CursorWidth < 0
    Or Style.CursorWidth > Core.UI_MAX_LAYOUT_VALUE
    Or Style.CursorHeight < 0
    Or Style.CursorHeight > Core.UI_MAX_LAYOUT_VALUE) Then
    Return False
End If
```

Keep **Then** on the same physical line as the closing parenthesis. Use four spaces for each continuation level and never use tab indentation.

Expressions, Calls, And Boundaries

The parser keeps the existing expression tree, precedence, semantic model, native output, Web output, source locations, and debugger statement boundaries.

Nested grouping and Else If

```
If ((IsVisible And IsEnabled) Or
    (IsSelected And IsAvailable)) Then
    Result = True
Else If (ItemCount = 0 Or
    IsClosing Or
    IsDisabled) Then
    Result = False
End If
```

Call argument continuation

Newlines may appear after the opening parenthesis, around arguments and commas, and before the closing parenthesis. Ordinary calls, qualified calls, built-in calls, and Call statements share the same parser rule.

```
Result = Menu.CalculateValue(
    MenuHandle,
    ItemIndex,
    NewValue
)

Call Menu.UpdateItem(
    MenuHandle,
    ItemIndex,
    Result
)
```

What remains invalid

```
If IsVisible Or
    IsEnabled Then
    Result = True
End If

Result = FirstValue +
    SecondValue
```

Routine declaration parameter lists and square-bracket array syntax remain line-oriented in this update. A newline between the closing parenthesis and **Then** is also invalid.

Permanent Source Readability Style

The source style is designed for students who read, hover, print, watch, and step through each value while learning.

Capitalization

Use Visual Basic-style initial capitalization for keywords and ordinary identifiers. Established constants may remain uppercase. Short interface labels use initial or title capitalization; complete sentences use normal English capitalization. Avoid all caps for ordinary keywords, variables, menu items, instructions, and prose.

Blank-line spacing

- Use exactly one blank line between logical groups; never use double or triple blank lines.
- Separate the final consecutive Module, Import, Dim, Call, or Unload statement from the following group.
- Put one blank line after a Function, Sub, or procedure declaration.
- Put one blank line before If, For, End For, Do, End Sub, and Loop, and after End If and Loop.
- Separate Option Explicit from the statements before and after it.
- A one-statement If block may omit extra blank lines directly inside the block.

Return named variables

A function assigns a literal, constant, member, call, comparison, or arithmetic result to a correctly typed variable before returning it. Keep one blank line between the final Return and End Function.

```
Function PointsFor(Row As Number) As Number
    Dim ReturnValue As Number

    ReturnValue = 70 - Row * 10
    Return ReturnValue

End Function
```

The repository formatter and checker is **scripts\format-smile-style.ps1**. The exact native/Web parity project is **examples\MultilineExpressionParity**.

Compatibility summary

- Existing one-line SMILE source remains valid.
- The lexer still emits newline tokens.
- The parser alone applies the balanced-parenthesis continuation scope.
- The AST and .smilelib formatVersion 5 remain unchanged.
- Visual Studio and the compiler use the same shared parser and language services.

For the complete current grammar and examples, use **docs\language\README.md** in the repository.